The GreatWall Signature Scheme v2

Authors Jiaqi Zheng, Tsinghua University
Tianhao Wang, Tsinghua University
Shiqi Hou, Tsinghua University
Guoxiao Liu, Tsinghua University
Xiaoyang Dong, Tsinghua University

Contact xiaoyangdong@tsinghua.edu.cn

Tuesday 30th June, 2026

Table of Contents

The GreatWall Signature Scheme v2 1

1 Introduction 4
2 Specification 5
2.1 Notations and Definitions 6
2.2 Specification of Pylon 7
2.3 Specifications of the GreatWall Signature Scheme 9
2.3.1 Key Generation. 14
2.3.2 Signature Signing. 14
2.3.3 Signature Verification. 23
2.3.4 Additional Functions. 27
3 Security Goals 29
3.1 Correctness 29
3.2 Unforgeability 29
3.3 One-Wayness of Pylon 30
3.4 Scope of the Security Claims 30
4 Features 31
4.1 A Modular Signature Design Centered on Pylon 31
4.2 Proof-Cost-Aware One-Way Function Design 32
4.3 Low-Round Structure with Security-Oriented Coupling 32
4.4 Parameter Selection Against Known Algebraic Shortcuts 33
4.5 Scalable Security Levels with a Uniform Architecture 34
4.6 Compact-Key and Signature-Size-Oriented Performance 34
5 Design Rationale 35
5.1 Choice of a Two-Round Architecture. 35
5.2 Choice of Field. 35
5.3 Choice of Nonlinear Layer: Inverse Mersenne Mapping. 36
5.3.1 Choice of the Exponents. 37
5.3.2 Linear Layer: Random Dense Matrix. 38
5.3.3 Nested Feed-forward Structure. 38
6 Security Evaluations 39
6.1 Brute-force Attack 39
6.2 Algebraic Attacks 39
6.3 Statistical Attacks 47
6.4 Quantum Attacks 48
6.5 Summary of Dedicated Attack Analysis 49
6.6 Formal Security Analysis 50
6.6.1 Security Models and Scope of the Generic Analysis 50
6.6.2 Generic Analysis of Pylon under the Ideal-Permutation Model 50

6.6.3 EUF-CMA Security of GreatWall in the Random Oracle Model 53
6.6.4 Post-Quantum Security Framework and Scope 58
7 Implementation and Performance 59
7.1 The Benchmarking Environments 59
7.2 Performance of GreatWall 59

1 Introduction

This document specifies **GreatWall**, a family of post-quantum digital signature schemes based solely on one-way function assumptions. GreatWall follows the *VOLE-in-the-Head* (VOLEitH) paradigm [8] and instantiates the public-key relation with a dedicated symmetric one-way function, called Pylon. The scheme is designed to provide existential unforgeability under adaptive chosen-message attacks while avoiding number-theoretic, lattice-based, code-based, or multivariate public-key assumptions. And it consists of four variants—GreatWall-128, GreatWall-192, GreatWall-256, and GreatWall-512—providing different levels of classical and post-quantum security, respectively.

The motivation behind GreatWall is the growing need for post-quantum signatures with compact public keys, predictable performance, and a scalable security path. Existing symmetric-key-based signature schemes have demonstrated that VOLEit techniques can yield practical post-quantum signatures, but their efficiency and size are strongly influenced by the cost of proving the underlying one-way-function relation. For example, the size of the extended witness and the number of nonlinear constraints directly affect signature size, signing time, and verification time. GreatWall aims at the trade-off between efficiency and security through a co-design of the signature layer and the underlying one-way function: the proof system is kept within the VOLEitH framework, while Pylon is constructed to minimize proof cost without introducing an algebraically fragile relation.

At a high level, a GreatWall signature is a non-interactive zero-knowledge proof of knowledge of a preimage of the public key under Pylon. The proof is obtained by applying the Fiat–Shamir transform [19] to a VOLE-based proof system. The signer commits to VOLE-related data using a GGM-tree-based batch all-but-one vector commitment, derives challenges through random oracles, and proves consistency of the Pylon relation through QuickSilver-style multiplication checks. Since linear operations are handled locally by the linearity of VOLE correlations, the dominant proof cost comes from the nonlinear part of Pylon. The security of the resulting scheme reduces entirely to the one-wayness of the underlying symmetric primitive and the soundness of the VOLEitH proof system, with no structured algebraic hardness assumption introduced. All the variants share the same construction; only the parameters of the one-way function and the proof system are adjusted across security levels, so that a single design instruments the entire family.

The dedicated one-way function, dubbed **Pylon**, is the core of the construction. Pylon is a symmetric primitive defined over a binary extension field, designed jointly with the VOLEitH proof system in order to keep the witness encoding compact and the consistency check structurally simple. Its central design feature is a *nested feed-forward* structure: an outer feed-forward branch spans the entire function and binds the secret input to the global input-output relation, while two inner feed-forward branches act on local sub-structures within the outer one. This nested arrangement is the result of a deliberate trade-off. The inner branches enrich the algebraic complexity of each round at almost no

additional multiplicative cost, while the outer branch ties the local algebraic obstructions together into a single global one-wayness relation. The construction therefore retains a small number of nonlinear constraints per round, which is essential for an efficient VOLEitH proof, while substantially raising the cost of any algebraic attack that attempts to peel off rounds independently. The detailed specifications and recommended parameters of Pylon are given in Sect. 2.2.

The security analysis of GreatWall follows a two-level approach. At the signature level, EUF-CMA security follows the FAEST/VOLEitH simulation-and-extraction framework of FAEST/VOLEitH proof system, and gives a conditional EUF-CMA reduction in the random-oracle model and its post-quantum analogue with the remaining instantiation requirements. At the primitive level, the required assumption is the preimage resistance of Pylon. This document supports the Pylon assumption through both a generic black-box analysis in the ideal-permutation model and dedicated cryptanalytic evaluations, including brute-force search, Gröbner-basis attacks, overdefined quadratic modeling, resultant attacks, subgroup-linearization attacks, statistical attacks, and so on.

GreatWall has been designed for robustness and scalability. It provides four parameter variants, namely GreatWall-128, GreatWall-192, GreatWall-256, and GreatWall-512. These variants share the same overall architecture and differ only in field size, exponent choices, and VOLEitH parameters. This uniform structure simplifies implementation and analysis, while allowing the scheme to scale naturally to higher security levels. Within the VOLEitH framework, this scalability translates directly into predictable signature sizes and predictable signing and verification cost across the entire family. The performance of GreatWall has been benchmarked against state-of-the-art post-quantum signature schemes built on symmetric primitives, with particular attention to leading VOLEitH-based designs. As a result, GreatWall is particularly well suited to applications that require long-term signature validity together with moderate signing latency, including firmware updates, code signing, and archival authentication.

The construction deliberately avoids assumptions vulnerable to Shor-type [32] quantum algorithms. The main generic quantum speedup relevant to Pylon is Grover search, which is accounted for in the parameter selection. Together with its compact-key structure, scalable parameterization, and proof-cost-aware one-way-function design, GreatWall is intended to be a practical candidate for long-term post-quantum authentication scenarios such as firmware signing, code signing, secure boot, software update distribution, and archival authentication.

2 Specification

This section presents a complete description of the GreatWall family, starting with the basic notations and definitions in Sect. 2.1, the specification of the one-way function Pylon in Sect. 2.2 and then an overview of the algorithms in Sect. 2.3.

Table 1: Notations

Notation	Description
$[n]$	The set $\{1, \ldots, n\}$ for any positive integer n.
$\mathrm{hw}(n)$	The Hamming weight of n, i.e., the number of non-zero bits in its binary representation.
$x \leftarrow a$	Assignment of the value a to the variable x.
$y \xleftarrow{\$} S$	Sampling y uniformly at random from the set S.
$+$	Addition in the corresponding algebraic structure, including extension fields and integers.
$\oplus$	Bitwise exclusive-OR operation.
$x^{(i)}$	The share of a secret x held by the i-th party in additive secret sharing, where the sum of all shares equals x.
λ	security parameter in classical settings.
n	Input or output bit length of S-boxes.
r	Total number of rounds of Pylon.
τ	Number of parallel repetitions required to achieve the target security level in the Non-Interactive Zero-Knowledge Proof of Knowledge (NIZKPoK) protocol.
N	Total number of parties in the proof system.
$\mathsf{negl}(\lambda)$	A negligible function in λ if it vanishes faster than the reciprocal of any polynomial in λ.
PPT	Probabilistic polynomial-time.

2.1 Notations and Definitions

Digital Signatures. A digital signature scheme consists of three PPT algorithms $(\mathsf{Gen}, \mathsf{Sign}, \mathsf{Verify})$:

- Key generation: $\mathsf{Gen}(1^\lambda)$ takes a security parameter λ as input and outputs a pair of public key and secret key (pk, sk).
- Signing: $\mathsf{Sign}(sk, M)$ produces a signature σ of the message $M \in \{0,1\}^*$ with the secret key sk.
- Verification: $\mathsf{Verify}(pk, M, \sigma)$ will return 1 if σ is valid for M and sk, and 0 otherwise.

A valid signature scheme should satisfy that for all honestly generated key pairs and all messages M, $\mathsf{Verify}(\mathsf{pk}, M, \mathsf{Sign}(\mathsf{sk}, M)) = 1$. Informally, a signature scheme can be considered as secure if no efficient adversary, after adaptively querying a signing oracle on chosen messages, can produce a valid signature on a new message with non-negligible probability. And Definition 1 captures this.

Definition 1. *Given a signature* $\Sigma = (\mathsf{Gen}, \mathsf{Sign}, \mathsf{Verify})$*, if for any PPT adversary* $\mathcal{A}$*, there always exists a negligible function* $\mathsf{negl}(\lambda)$ *such that*

$$\begin{aligned} \Pr\Big[&(pk, sk) \leftarrow \mathsf{Gen}(1^\lambda), \\ &(M', \sigma') \leftarrow \mathcal{A}^{\mathsf{Sign}(sk,\cdot)}(pk, 1^\lambda), \\ &M' \notin Q,\ \mathsf{Verify}(pk, M', \sigma') = 1\Big] < \mathsf{negl}(\lambda). \end{aligned} \tag{1}$$

where $\mathcal{A}^{\mathsf{Sign}(sk,\cdot)}(pk, 1^\lambda)$ *denotes that* $\mathcal{A}$ *has access to the oracle* $\mathsf{Sign}(sk, \cdot)$ *with* (pk, sk) *already generated,* Q *is the set of the queries made by* $\mathcal{A}$*. Then we can say that the signature* Σ *is secure against existential unforgeability under chosen message attack (EUF-CMA), or EUF-CMA secure.*

In the post-quantum setting, the adversary $\mathcal{A}$ would be modeled as a quantum polynomial-time algorithm, while the accessible signing oracle remains classical. The existence of secure digital signature schemes is fundamentally tied to the existence of one-way functions (OWFs). Specifically, it has been theoretically proven by Rompel that OWFs are both a necessary and sufficient condition for constructing secure signatures [31].

One-Way functions. A function $f : \{0,1\}^* \to \{0,1\}^*$ is an OWF only when two conditions can be satisfied, $f(x)$ is polynomial-time computable for a randomly chosen input $x \xleftarrow{\$} \{0,1\}^*$, and computationally infeasible for any polynomial-time adversary to recover x with non-negligible probability. OWFs can be instantiated from symmetric cryptographic primitives such as block ciphers, permutations, or hash-function-based constructions. In contrast to number-theoretic assumptions, the security of symmetric primitives in the presence of quantum adversaries is comparatively well understood. Known quantum algorithms, most notably Grover's algorithm [23], provide at most a quadratic speedup for generic attacks.

2.2 Specification of Pylon

A new one-way function, Pylon, serves as the core building block of GreatWall. The name reflects its role as a simple yet load-bearing component supporting the overall VOLEitH signature scheme. This new one-way function is constructed based on an *iterative, nested feed-forward structure*, and consists of a nonlinear operation S and a constant addition over large field $\mathbb{F}_{2^n}$, together with a linear layer.

Given input/output size n and exponents $(e_1, e_2) \in \mathbb{Z}^2$, Pylon: $\{0,1\}^n \times \mathbb{F}_{2^n} \to \mathbb{F}_{2^n}$ is defined as

$$\begin{cases} x_1 = S_1(M_1(pt) + \gamma_1), \\ x_2 = S_2(M_2(x_1 + pt) + \mathrm{iv}_{\mathrm{owf}}), \\ ct = M_3(x_2) + M_2(x_1 + pt) + \mathrm{iv}_{\mathrm{owf}} + pt + \gamma_2. \end{cases} \tag{2}$$

And the overview of our construction is captured in Figure 1.

Nonlinear Components.

The nonlinear layer $S : \mathbb{F}_{2^n} \to \mathbb{F}_{2^n}$, denoted as $\mathrm{Mer}[e]^{-1}$, is instantiated as an inverse Mersenne mapping,

$$S(x) = x^{\bar{e}}, \qquad \bar{e} = (2^e - 1)^{-1} \bmod (2^n - 1), \tag{3}$$

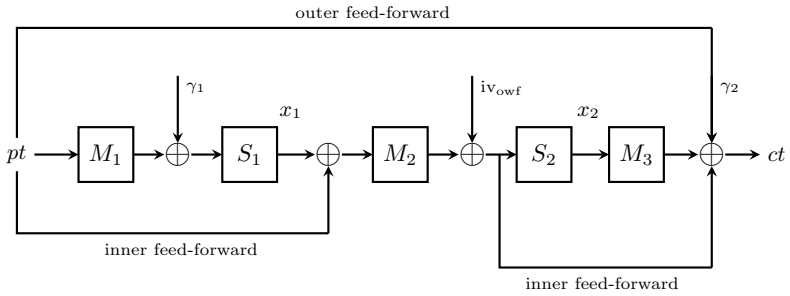


Fig. 1: Overview of the proposed one-way function construction.

where $\gcd(e, n) = 1$. This condition implies $\gcd(2^e - 1, 2^n - 1) = 1$, and therefore the map $x \mapsto x^{2^e-1}$ is a permutation over $\mathbb{F}_{2^n}^*$. Consequently, the inverse exponent $\bar{e} = (2^e - 1)^{-1} \bmod (2^n - 1)$ defines a permutation over $\mathbb{F}_{2^n}$. This choice prevents a high abundance of low-degree relations while avoiding potential overdefined algebraic structures.

Linear Components.

Pylon includes three types of linear components: round constants γ_i, linear diffusion layer $M_i \in (\mathbb{F}_2)^{n\times n}$, and feed-forward structures. To facilitate the implementation and analysis, M_i can be equivalently represented as a linearized polynomial $L_i \in \mathbb{F}_{2^n}[X]$ of the form $L_i(X) = \sum_{j=0}^{n-1} a_j^{(i)} X^{2^j}$, where the coefficients $a_j^{(i)} \in \mathbb{F}_{2^n}$. Throughout our construction, we require that $a_j^{(i)} \neq 0$ for all $j \in \{0, \ldots, n-1\}$, to ensure that the polynomial in use is of maximum degree and as dense as possible. This design choice aims to maximize diffusion and to avoid sparse linear structures that could potentially facilitate algebraic or linear cryptanalytic attacks.

Remark on Matrix and Round Constants Generations. We adopt a parameter generation method inspired by the Rain design [16]. For clarity, this section describes the matrices and round constants generation procedure for a single round and omits the round index, and denote M_i and its corresponding linearized polynomial L_i by M and L.

The procedure can be divided into three steps:

1. *Sampling a candidate linear layer L.* We sample the coefficients of a candidate linearized polynomial

$$L(X) = \sum_{j=0}^{n-1} a_j X^{2^j}$$

from a pseudo-random bit stream and parse them as elements of $\mathbb{F}_{2^n}$. If any coefficient a_j is zero, the sample is rejected and resampled, as the design

mandates a dense linear layer. Once accepted, the polynomial L uniquely defines the corresponding linear transformation $M \in (\mathbb{F}_2)^{n\times n}$.

2. *Checking that L is a permutation and that L^{-1} is also dense.* We build the Dickson matrix D_L of L by

$$D_L = \begin{pmatrix} a_0 & a_1 & \cdots & a_{n-1} \\ a_{n-1}^2 & a_0^2 & \cdots & a_{n-2}^2 \\ \vdots & \vdots & \vdots & \vdots \\ a_1^{2^{n-1}} & a_2^{2^{n-1}} & \cdots & a_0^{2^{n-1}} \end{pmatrix}.$$

Then L is a permutation if and only if D_L is invertible [27]. If $\det(D_L) = 0$, we reject and resample. If $\det(D_L) \neq 0$, we compute L^{-1} from cofactors of D_L. This gives a linearized form

$$L^{-1}(X) \;=\; \frac{1}{\det(D_L)} \sum_{i=0}^{n-1} \tilde{a}_i X^{2^i},$$

where $\tilde{a}_i$ comes from the cofactor values of D_L. We reject the sample if any $\tilde{a}_i = 0$. So both L and L^{-1} have all nonzero coefficients with degree of 2^{n-1}.

3. *Building the binary matrix M.* After we fix L, we convert it to the matrix $M \in (\mathbb{F}_2)^{n\times n}$ by using a basis and a dual basis as shown in [13,37]. Let $\{\beta_1, \ldots, \beta_n\}$ be an $\mathbb{F}_2$-basis of $\mathbb{F}_{2^n}$ and let $\{\beta'_1, \ldots, \beta'_n\}$ be the dual basis such that $\mathrm{Tr}(\beta_i \beta'_j) = \delta_{i,j}$, where $\mathrm{Tr} : \mathbb{F}_{2^n} \to \mathbb{F}_2$ is the trace map. Then we define the matrix entries by

$$M_{i,j} \;=\; \mathrm{Tr}\big(\beta'_i \cdot L(\beta_j)\big).$$

This derives the $\mathbb{F}_2$-linear map used in the round.

Round constants are generated in a similar way from the same pseudo-random stream and parsed as elements of $\mathbb{F}_{2^n}$. We generate the round constants with the SHAKE256 extendable-output function [30]. We initialize SHAKE256 with the UTF-8 string `GreatWall-N` where N is replaced by the state size in bits. For each round i, we squeeze n bits from SHAKE256 and interpret them as an element $\gamma_i \in \mathbb{F}_{2^n}$. We also use the same SHAKE256 stream to sample the values needed for the linear layer generation above, such as the coefficients $a_j \in \mathbb{F}_{2^n}$. These constants are public and fixed, and they are only used to avoid symmetries across rounds. So using SHAKE256 [30] is reasonable, since this step does not need to provide security strength.

Recommended Parameters. The concrete instances and recommended parameters of Pylon are listed in Table 2.

2.3 Specifications of the GreatWall Signature Scheme

VOLE-in-the-Head. VOLE-in-the-Head (VOLEitH) [8] is a general framework for transforming VOLE-based zero-knowledge proofs [7,36,40] into publicly verifiable proofs, and hence into digital signatures via the Fiat–Shamir transform

Variant	Parameters (λ, n, e_1, e_2)	Extension Field	Irreducible Polynomial
Pylon-128	$(128, 137, 70, 75)$	$\mathbb{F}_{2^{137}}$	$X^{137} + X^{21} + 1$
Pylon-160	$(160, 163, 79, 86)$	$\mathbb{F}_{2^{163}}$	$X^{163} + X^7 + X^6 + X^3 + 1$
Pylon-192	$(192, 197, 94, 105)$	$\mathbb{F}_{2^{197}}$	$X^{197} + X^{21} + X^2 + X + 1$
Pylon-256	$(256, 263, 129, 136)$	$\mathbb{F}_{2^{263}}$	$X^{263} + X^{93} + 1$
Pylon-512	$(512, 521, 248, 273)$	$\mathbb{F}_{2^{521}}$	$X^{521} + X^{32} + 1$

Table 2: Recommended sets of parameters of Pylon.

[19]. At a high level, the prover first commits to randomly generated VOLE-related data without knowing the final global key Δ, then a challenge determines Δ, and finally the prover opens the commitments in a way that allows the verifier to reconstruct the corresponding VOLE keys and check the proof.

VOLE correlations. Let $\mathbb{F}_{2^k}$ be a finite field. A length-ℓ VOLE correlation consists of values

$$(u_i, v_i, q_i, \Delta) \in \mathbb{F}_2 \times \mathbb{F}_{2^k} \times \mathbb{F}_{2^k} \times \mathbb{F}_{2^k}, \qquad i = 0, \ldots, \ell - 1,$$

such that $q_i = u_i \cdot \Delta - v_i$. Here, the prover holds (u_i, v_i), whereas the verifier holds (Δ, q_i). Since the relation is linear in Δ, VOLE can be viewed as a linearly homomorphic commitment to the bits u_i. This property is the main reason VOLE is well suited for efficient zero-knowledge proofs.

Small-field instantiations and concatenation. In practice, directly working over $\mathbb{F}_{2^\lambda}$ is too expensive. VOLEitH therefore instantiates several smaller VOLE correlations in parallel and concatenates them to obtain a large-field correlation over $\mathbb{F}_{2^\lambda}$ [9]. Concretely, one chooses a repetition parameter τ and defines

$$k_0 = \left\lceil \frac{\lambda}{\tau} \right\rceil, \qquad k_1 = \left\lfloor \frac{\lambda}{\tau} \right\rfloor, \qquad \tau_0 = \lambda \bmod \tau, \qquad \tau_1 = \tau - \tau_0,$$

so that $\tau_0 k_0 + \tau_1 k_1 = \lambda$. Then τ_0 instances over $\mathbb{F}_{2^{k_0}}$ and τ_1 instances over $\mathbb{F}_{2^{k_1}}$ are generated and concatenated column-wise to form one VOLE relation over $\mathbb{F}_{2^\lambda}$.

All-but-one vector commitments from GGM trees. The main tool enabling delayed VOLE generation is an all-but-one vector commitment based on a GGM tree [21,9]. Starting from a root seed r, a length-doubling pseudorandom generator recursively expands the seed into a complete binary tree with $N = 2^d$ leaves. Each leaf key is hashed to obtain $(\mathsf{sd}_j, \mathsf{com}_j), \quad j = 0, \ldots, N-1$, where sd_j is the leaf seed used later for VOLE generation and com_j is its local commitment. The overall commitment is

$$h_{\mathsf{com}} = H_1(\mathsf{com}_0, \ldots, \mathsf{com}_{N-1}).$$

To open all positions except one hidden index $j^\star$, the prover sends the copath of $j^\star$ together with $\mathsf{com}_{j^\star}$. This allows the verifier to reconstruct all other leaves and recompute the global commitment, while the unopened leaf remains hidden.

From committed seeds to VOLE. Let the leaf seeds be expanded into vectors $r_0, r_1, \ldots, r_{N-1} \in \mathbb{F}_2^{\hat{\ell}}$. A small-field VOLE instance is then derived by defining

$$u = \sum_{j=0}^{N-1} r_j, \qquad v = \sum_{j=0}^{N-1} j \cdot r_j,$$

where j is interpreted as an element of $\mathbb{F}_{2^k}$. If the verifier later learns all leaf data except at index Δ, it can compute

$$q = \sum_{j=0}^{N-1} (\Delta - j) \cdot r_j = \Delta \cdot \sum_{j=0}^{N-1} r_j - \sum_{j=0}^{N-1} j \cdot r_j = \Delta \cdot u - v.$$

Hence, opening all but one leaf directly yields a VOLE correlation whose challenge value is the unopened index.

Witness masking and chosen VOLE. The VOLE values generated above are initially random and independent of the witness. To bind them to the witness w of the underlying one-way-function relation, the prover masks the witness with the random VOLE secret and publishes $d = w - u$. Equivalently, it writes $w = u + d$. Since the verifier already holds a VOLE key of the form $q_i = u_i\Delta - v_i$, it can update this key using the public mask d_i as

$$q_i' = q_i + d_i\Delta = (u_i\Delta - v_i) + d_i\Delta = (u_i + d_i)\Delta - v_i = w_i\Delta - v_i.$$

Therefore, after applying the mask, the verifier obtains a VOLE authentication of the actual witness bit w_i. In this way, the original random VOLE is transformed into a *chosen VOLE* bound to the witness. This step is crucial, because the subsequent proof system no longer works with a random hidden vector u, but with the witness w itself.

QuickSilver-based proof stage. Once the chosen VOLE correlations for the witness are available, VOLEitH is typically combined with the QuickSilver proof system to prove that the witness satisfies the target circuit [40]. Consider an arithmetic or Boolean circuit C with multiplication gates. For each input wire and each multiplication-gate output wire, the prover and verifier hold corresponding VOLE-authenticated values. Linear operations can be handled locally thanks to the linearity of VOLE, so only multiplication constraints require an explicit check.

For a multiplication gate indexed by (α, β, γ), where the intended relation is $w_\gamma = w_\alpha w_\beta$, the verifier computes $b_i = q_\alpha q_\beta - q_\gamma\Delta$. Substituting the VOLE relations

$$q_\alpha = w_\alpha\Delta - v_\alpha, \qquad q_\beta = w_\beta\Delta - v_\beta, \qquad q_\gamma = w_\gamma\Delta - v_\gamma,$$

one obtains

$$\begin{aligned} b_i &= (w_\alpha \Delta - v_\alpha)(w_\beta \Delta - v_\beta) - (w_\gamma \Delta - v_\gamma)\Delta \\ &= v_\alpha v_\beta + (w_\alpha v_\beta + w_\beta v_\alpha - v_\gamma)\Delta + (w_\alpha w_\beta - w_\gamma)\Delta^2. \end{aligned}$$

If the multiplication constraint is satisfied, i.e. $w_\gamma = w_\alpha w_\beta$, then the quadratic term vanishes and b_i becomes an affine function of Δ:

$$b_i = a_{0,i} + a_{1,i}\Delta,$$

where $a_{0,i} = v_\alpha v_\beta, a_{1,i} = w_\alpha v_\beta + w_\beta v_\alpha - v_\gamma$. Thus, to prove correctness of the multiplication gate, it suffices for the prover to provide information allowing the verifier to check this affine relation.

To reduce communication, QuickSilver does not send $(a_{0,i}, a_{1,i})$ for every multiplication gate separately. Instead, the verifier samples a random challenge χ, and the prover forms a random linear combination

$$a_0 = \sum_i \chi^i a_{0,i}, \qquad a_1 = \sum_i \chi^i a_{1,i},$$

while the verifier computes $b = \sum_i \chi^i b_i$. The verifier then only needs to check the single equality $b = a_0 + a_1\Delta$.

Batch All-but-one Vector Commitment. Let λ be the security parameter and let $\mathcal{M}$ be a message space. For an integer τ, let $(N_0, \ldots, N_{\tau-1})$, be the lengths of τ committed vectors. We use $[n] = \{0, \ldots, n-1\}$. A batch all-but-one vector commitment (BAVC) scheme [6] is a tuple of algorithms

$$\mathsf{BAVC} = (\mathsf{Commit}, \mathsf{Open}, \mathsf{Verify})$$

defined as follows.

$\mathsf{Commit}() \to (\mathsf{com}, \mathsf{decom}, (m_1^{(\alpha)}, \ldots, m_{N_\alpha}^{(\alpha)})_{\alpha\in[\tau]})$:
output a commitment com with opening information decom for messages

$$(m_1^{(\alpha)}, \ldots, m_{N_\alpha}^{(\alpha)})_{\alpha\in[\tau]} \in \mathcal{M}^{N_1+\cdots+N_\tau}.$$

$\mathsf{Open}(\mathsf{decom}, I) \to \mathsf{decom}_I \vee \perp$:
On input an opening information decom and the index vector $I \in [N_1] \times \cdots \times [N_\tau]$, output $\perp$ or an opening decom_I for I. Here, $I = (i_1, \ldots, i_\tau)$ specifies the unopened position in each committed vector.

$\mathsf{Verify}(\mathsf{com}, \mathsf{decom}_I, I) \to ((m_j^{(\alpha)})_{j\in[N_\alpha]\setminus\{i_\alpha\}})_{\alpha\in[\tau]} \vee \perp$:
Given a commitment com, an opening decom_I for an index vector I, either output all messages

$$((m_j^{(\alpha)})_{j\in[N_\alpha]\setminus\{i_\alpha\}})_{\alpha\in[\tau]}$$

and accept the opening, or output $\perp$ and reject the opening.

Correctness with aborts. BAVC is correct with aborts if, for every index vector

$$I \in [N_0] \times \cdots \times [N_{\tau-1}],$$

the following experiment outputs true except with negligible probability:

$$(\mathsf{com}, \mathsf{decom}, M) \leftarrow \mathsf{Commit}(),$$

where

$$M = \left(m_0^{(\alpha)}, \ldots, m_{N_\alpha - 1}^{(\alpha)}\right)_{\alpha \in [\tau]}.$$

For every

$$\mathsf{decom}_I \leftarrow \mathsf{Open}(\mathsf{decom}, I),$$

it holds that either $\mathsf{decom}_I = \perp$, or

$$\mathsf{Verify}(\mathsf{com}, \mathsf{decom}_I, I) = \left(\left(m_j^{(\alpha)}\right)_{j \in [N_\alpha] \setminus \{i_\alpha\}}\right)_{\alpha \in [\tau]}.$$

Thus, whenever the opening does not abort, the verifier can recover all opened messages consistently with the original commitment, while the entries $\left(m_{i_\alpha}^{(\alpha)}\right)_{\alpha \in [\tau]}$ remain unopened.

GGM-based instantiation. In VOLE-in-the-Head based signatures, BAVC is typically instantiated from a GGM tree. The committed messages are pseudorandom seeds derived from a root seed by repeatedly applying a length-doubling pseudorandom generator. Instead of using τ independent trees, the optimized construction uses a single interleaved GGM tree with $L = \sum_{\alpha=0}^{\tau-1} N_\alpha$ leaves. Each pair (α, j), corresponding to the j-th entry of the α-th vector, is mapped to a unique leaf position

$$\pi(\alpha, j) \in [L - 1, 2L - 2].$$

The leaf at position $\pi(\alpha, j)$ determines the seed $m_j^{(\alpha)}$ and its leaf commitment. The vector-level commitments are obtained by hashing the leaf commitments inside each vector, and the final commitment com is obtained by hashing all vector-level commitments.

Given an index vector $I = (i_0, \ldots, i_{\tau-1})$, the opening algorithm keeps the τ leaves

$$\pi(0, i_0), \ldots, \pi(\tau - 1, i_{\tau-1})$$

unopened and reveals enough internal tree nodes to reconstruct all remaining leaves. Since different opening paths may merge in the single interleaved tree, fewer internal nodes need to be sent compared with opening τ independent GGM trees. To keep the decommitment size fixed, the scheme sets an opening threshold T_{open}. If more than T_{open} internal nodes are required, then $\mathsf{Open}(\mathsf{decom}, I) = \perp$.

Grinding. In the non-interactive signature scheme, the index vector I is derived from a Fiat–Shamir challenge. Let w_{grind} be the grinding parameter. The signer computes

$$\mathsf{chall}_3 = H(\mathsf{transcript} \,\|\, \mathsf{ctr}) \in \{0,1\}^{\lambda},$$

where ctr is a counter initialized to zero. The first $\lambda - w_{\text{grind}}$ bits of chall_3 are decoded into the index vector I, while the remaining w_{grind} bits are required to be zero. If this condition is not satisfied, or if $\mathsf{Open}(\mathsf{decom}, I) = \perp$, the signer increments ctr and recomputes the challenge. The final counter value is included in the signature, so the verifier can recompute chall_3, check the grinding condition, decode the same index vector I, and verify the BAVC opening.

Next the whole description of the GreatWall signature scheme will be given, consisting of key generation, signing, and verification algorithms.

2.3.1 Key Generation.

The key generation takes as input a randomly generated pt and outputs a public key $(\text{iv}_{\text{owf}}, ct)$ and a secret key pt such that $ct = \text{Pylon}(\text{iv}_{\text{owf}}, pt)$. The algorithm is given in algorithm 1.

Algorithm 1: Pylon.KeyGen(1^{λ})

repeat
  $pt \xleftarrow{\$} \text{GF}(2^n)$
  $\text{iv}_{\text{owf}} \xleftarrow{\$} \{0,1\}^n$
  $\alpha_1 := \text{SHAKE}(\text{iv}_{\text{owf}} \| 0)$
  $\alpha_2 := \text{SHAKE}(\text{iv}_{\text{owf}} \| 1)$
  $(ct, x_1, x_2) := \mathsf{Pylon}(\text{iv}_{\text{owf}}, pt)$
until $\langle \alpha_1, x_1 \rangle \neq 0 \ \wedge \ \langle \alpha_2, x_2 \rangle \neq 0$
$pk := (\text{iv}_{\text{owf}}, ct)$
$sk := pt$
return (pk, sk)

2.3.2 Signature Signing.

The signing algorithm of the GreatWall signature is given in algorithm 2. It takes as input the secret and public keys $(pt, (\text{iv}_{\text{owf}}, ct))$ and a message m and outputs a signature σ. The signing algorithm utilizes several procedures defined in earlier sections, and the signature generation is executed through four main phases, which are delineated by the derivation of these challenges.

Algorithm 2: GreatWall.Sign(m,pk,sk)

$pt \leftarrow sk$
$\mathrm{iv}_{owf}, ct \leftarrow pk$
$\mathrm{ctr} \leftarrow 0$ // 32-bit int
$\rho \xleftarrow{\$} \{0,1\}^{\lambda}$ // Signature randomness
$\mu := H_1(ct \| m; 2\lambda)$
$(r, \mathrm{iv}_{\mathrm{vole}}) := H_3(pt \| \mu \| \rho)$ // $r \in \{0,1\}^{\lambda}$, $\mathtt{iv_{vole}} \in \{0,1\}^{128}$
// Commitment and VOLE Derandomization
$(\mathrm{com}, \mathrm{decom}, c_1, \ldots, c_{\tau-1}, \mathbf{u}, \mathbf{V}) := \mathsf{VOLECommit}(r, \mathrm{iv}_{\mathrm{vole}}, \ell + 2\lambda + B)$
$\mathsf{chall}_1 := H_2^1(\mu \| \mathrm{com} \| c_1 \| \cdots \| c_{\tau-1} \| \mathrm{iv}_{\mathrm{vole}}; 5\lambda + 64)$
// VOLE consistency check and witness commitment
$\widetilde{\mathbf{u}} := \mathsf{VOLEHash}(\mathsf{chall}_1, \mathbf{u}) \in \{0,1\}^{\lambda+B}$
$\widetilde{\mathbf{V}} := \mathsf{VOLEHash}(\mathsf{chall}_1, \mathbf{V}) \in \{0,1\}^{(\lambda+B)\times\lambda}$
$h_V := H_1(\widetilde{\mathbf{V}})$ // hash column-wise
$(ct, x_1, x_2) := \mathsf{Pylon}(\mathrm{iv}_{owf}, pt)$
$w := (pt, x_1)$
$\mathbf{d} := \mathbf{w} \oplus \mathbf{u}_{[0..\ell)}$ // Mask extended witness
$\mathsf{chall}_2 := H_2^2(\mathsf{chall}_1 \| \widetilde{\mathbf{u}} \| h_V \| \mathbf{d}; 3\lambda + 64)$
// Pylon proof
$\mathbf{u} := \mathbf{u}_{[0..\ell+\lambda)}$
$\mathbf{V} := \mathbf{V}_{[0..\ell+\lambda)}$ // drop last $\lambda + B$ rows
$(\widetilde{a}, \widetilde{b}) := \mathsf{ZK.OWFProve}(\mathbf{w}, \mathbf{u}, \mathbf{V}, ct, \mathrm{iv}_{owf}, \mathsf{chall}_2)$
$\mathrm{ctr} \leftarrow -1$
repeat
  $\mathrm{ctr} \leftarrow \mathrm{ctr} + 1$
  $\mathsf{chall}_3 \leftarrow H_2^3(\mathsf{chall}_2 \| \widetilde{a} \| \widetilde{b} \| \mathrm{ctr}; \lambda)$
  if $\mathrm{chall}_3[\lambda - w_{grind} : \lambda] \neq 0^{w_{grind}}$ **then**
    continue
  $I \leftarrow \mathsf{DecodeAllChall3}(\mathrm{chall}_3[0 : \lambda - w_{\mathrm{grind}}])$
  $\mathrm{decom}_I \leftarrow \mathsf{BAVC.Open}(\mathrm{decom}, I)$
until $\mathrm{decom}_I \neq \perp$
// Build VOLE challenge and open Vector Commitments
return $\sigma := \big((c_i)_{i\in[1..\tau)}, \widetilde{\mathbf{u}}, \mathbf{d}, \widetilde{a}, \mathrm{decom}_I, \mathrm{chall}_3, \mathrm{iv}_{vole}, \mathrm{ctr}\big)$

Phase 1. The signing procedure begins by setting an integer counter ctr to zero, which will be utilized later during the "grind check". Following this, the algorithm samples a random value $\rho \in \{0,1\}^{\lambda}$ to serve as the signature randomness. Next, it hashes the public key pk and the message m together via H_1 to output a 2λ-bit digest μ. The signer then processes the secret key pt, the digest μ, and ρ through H_3. This operation generates a λ-bit PRG seed r alongside an intermediate 128-bit string $\mathrm{iv}_{\mathrm{vole}}$. Using the pair $(r, \mathrm{iv}_{\mathrm{vole}})$, the signer calls the VOLECommit function to simultaneously commit to all VOLE instances. This single execution packages the small-VOLE vectors into a comprehensive GGM-

based batch vector commitment. It outputs a primary commitment com, a bulk decommitment structure decom that allows opening most seeds while keeping exactly one hidden per vector, a series of correction vectors $c_1, \ldots, c_{\tau-1}$ designed to align every VOLE instance to a shared secret, and the global VOLE secret u along with the MAC tags V. Finally, the signer extracts the first challenge chall_1 by hashing μ, com, the correction vectors $\{c_i\}$, and $\mathrm{iv}_{\mathrm{vole}}$ via H_2^1, requesting an output of $5\lambda + 64$ bits.

Algorithm 3: $\mathsf{VOLECommit}(r, \mathrm{iv}_{\mathrm{vole}}, \widehat{\ell})$

// Set VOLE vector sizes
for $i \in [0..\tau)$ **do**
  $\kappa_i := \begin{cases} k_0, & i < \tau_0, \\ k_1, & i \geq \tau_0 \end{cases}$
  $N_i := 2^{\kappa_i}$
// Batch all-but-one vector commitment
$(\mathrm{com}, \mathrm{decom}, (\mathrm{sd}_{i,0}, \ldots, \mathrm{sd}_{i,N_i-1})_{i\in[0..\tau)}) :=$ BAVC.Commit$(r, \mathrm{iv}_{\mathrm{vole}}, (N_i)_{i\in[0..\tau)})$
// Convert committed seeds to small VOLE correlations
for $i \in [0..\tau)$ **do**
  $(\mathbf{u}_i, \mathbf{v}_{i,0}, \ldots, \mathbf{v}_{i,\kappa_i-1}) :=$ ConvertToVOLE$(\mathrm{sd}_{i,0}, \ldots, \mathrm{sd}_{i,N_i-1}, \mathrm{iv}_{\mathrm{vole}}, \widehat{\ell})$
  $\mathbf{V}_i := [\mathbf{v}_{i,0} \ \cdots \ \mathbf{v}_{i,\kappa_i-1}] \in \mathbb{F}_2^{\widehat{\ell}\times\kappa_i}$
// Concatenate small VOLE tags into one large VOLE tag matrix
$\mathbf{V} := [\mathbf{V}_0 \ \cdots \ \mathbf{V}_{\tau-1}] \in \mathbb{F}_2^{\widehat{\ell}\times\lambda}$
// Use the first VOLE secret as the common secret
$\mathbf{u} := \mathbf{u}_0$
// Correction values for enforcing the same VOLE secret
for $i \in [1..\tau)$ **do**
  $c_i := \mathbf{u} \oplus \mathbf{u}_i$
return $(\mathrm{com}, \mathrm{decom}, c_1, \ldots, c_{\tau-1}, \mathbf{u}, \mathbf{V})$

Phase 2. In the second phase, the signer computes $\tilde{V}$ and $\tilde{u}$ to prepare for the VOLE consistency check. This is achieved by invoking VOLEHash on inputs (V, chall_1) and (u, chall_1), respectively. Then h_V is computed to generate the second challenge. Subsequently, the algorithm runs ExtendWitness using pt and the output of the first S-box to construct the extended witness w. To properly commit to w, the signer computes the difference $d := w \oplus u_{[0..l)}$, effectively masking w with a one-time pad derived from u. A second challenge chall_2, is then derived by hashing the concatenation of chall_1, $\tilde{u}$, h_V, and d.

Phase 3. During the third phase, the signer generates the QuickSilver proof. Initially, it truncates u and V to the first $l + \lambda$ rows. It then executes the zero-knowledge proof procedure ZK.OWFProve with inputs w, u, V, ct, and chall_2.

This function returns $\tilde{a}$, and $\tilde{b}$. The signer then enters a grinding loop: it computes the third challenge $\mathsf{chall}_3 \leftarrow \mathsf{H}_2^3(\mathsf{chall}_2 \| \widetilde{a} \| \widetilde{b} \| \mathsf{ctr}; \lambda)$, and checks if the last w_{grind} bits of chall_3 are zero. If not, it increments ctr and repeats. Once a valid chall_3 is found, the signer decodes the first $\lambda - w_{\mathsf{grind}}$ bits of chall_3 into an index vector I using DecodeAllChall3, and calls BAVC.Open with the decommitment decom and index I to generate a partial decommitment decom_I. If BAVC.Open returns $\perp$ (the opening size exceeds the threshold T_{open}), the loop increments ctr and restarts the challenge computation.

Algorithm 4: $\mathsf{BAVC.Commit}(r, \mathrm{iv}_{\mathrm{vole}}, (N_i)_{i \in [0..\tau)})$

```
// Compute the number of leaves in the single batched tree
L := ∑_{i=0}^{τ-1} N_i
N_max := max_{i∈[0..τ)} N_i
// Define the interleaving between virtual commitments and
   tree leaves
t := 0
for j ∈ [0..N_max) do
    for i ∈ [0..τ) do
        if j < N_i then
            π(t) := (i, j)
            π^{-1}(i, j) := t
            t := t + 1
// Expand one GGM tree with L leaves
K_0 := r
for a ∈ [0..L − 1) do
    (K_{2a+1}, K_{2a+2}) := PRG(K_a ‖ iv_vole; 2λ)
T := (K_0, K_1, ..., K_{2L−2})
// Derive committed seeds and leaf commitments
for t ∈ [0..L) do
    (i, j) := π(t)
    (sd_{i,j}, leafcom_{i,j}) := H_0(K_{L−1+t} ‖ iv_vole)
for i ∈ [0..τ) do
    h_i := H_1(leafcom_{i,0} ‖ leafcom_{i,1} ‖ ··· ‖ leafcom_{i,N_i−1})
hcom := H_1(h_0 ‖ h_1 ‖ ··· ‖ h_{τ−1})
decom := (T, (leafcom_{i,j})_{i∈[0..τ), j∈[0..N_i)})
return (hcom, decom, (sd_{i,0}, ..., sd_{i,N_i−1})_{i∈[0..τ)})
```

Phase 4. In the final phase, the signer assembles the complete signature once the grinding check and partial commitment opening both succeed. The signature σ is constructed as a tuple containing: the correction vectors $c_1, \ldots, c_{\tau-1}$, the hashed VOLE secret $\widetilde{\mathbf{u}}$, the masked witness $\mathbf{d}$, the zero-knowledge proof tags $\widetilde{a}$, the partial decommitment decom_I, the final challenge chall_3, the intermediate

Algorithm 5: $\mathsf{ConvertToVOLE}(\mathrm{sd}_0, \ldots, \mathrm{sd}_{N-1}, \mathrm{iv}_{\mathrm{vole}}, \widehat{\ell})$

```
1  d := log_2 N                                                    // N = 2^d
2  // Expand committed seeds into length-ℓ̂ bit vectors
3  for i ∈ [0..N) do
4      r_{0,i} := PRG(sd_i, iv_vole; ℓ̂)
5  // Initialize the d VOLE tag columns
6  for j ∈ [0..d) do
7      v_j := 0^{ℓ̂}
8  // Compress N expanded vectors into one secret vector and d tag
       columns
9  for j ∈ [0..d) do
10     for i ∈ [0..N/2^{j+1}) do
11         v_j := v_j ⊕ r_{j,2i+1}
12         r_{j+1,i} := r_{j,2i} ⊕ r_{j,2i+1}
13 u := r_{d,0}
14 return (u, v_0, ..., v_{d-1})
```

initialization vector $\mathrm{iv}_{\mathrm{vole}}$, and the grinding counter ctr. The algorithm outputs this tuple σ as the valid signature on message m.

Algorithm 6: $\mathsf{VOLEHash}(\mathsf{chall}, \mathbf{x})$

// $\mathbf{x} \in \{0,1\}^{\ell+2\lambda+B}$
$\mathbf{x}_0 := \mathbf{x}_{[0..\ell+\lambda)}$
$\mathbf{x}_1 := \mathbf{x}_{[\ell+\lambda..\ell+2\lambda+B)}$
$(r_0, r_1, r_2, r_3, s, t) := \mathsf{chall}$
// $r_0, r_1, r_2, r_3, s \in \{0,1\}^{\lambda}, \ \ t \in \{0,1\}^{64}$
for $i \in [0..4)$ **do**
  $r_i := \mathrm{ToField}(r_i, \lambda)$
$s := \mathrm{ToField}(s, \lambda)$
$t := \mathrm{ToField}(t, 64)$
$\ell' := \lambda \cdot \left\lceil \frac{\ell+\lambda}{\lambda} \right\rceil$
$\mathbf{x}_0 := \mathbf{x}_0 \| 0^{\ell'-(\ell+\lambda)}$
$\widehat{\mathbf{y}} := \mathrm{ToField}(\mathbf{x}_0, \lambda)$
$\overline{\mathbf{y}} := \mathrm{ToField}(\mathbf{x}_0, 64)$
$h_0 := 0$
for $i \in [0..\ell'/\lambda)$ **do**
  $h_0 := h_0 + s^{\ell'/\lambda-1-i} \cdot \widehat{\mathbf{y}}[i]$
$h_1 := 0$
for $i \in [0..\ell'/64)$ **do**
  $h_1 := h_1 + t^{\ell'/64-1-i} \cdot \overline{\mathbf{y}}[i]$
$h_1' := \mathrm{ToField}\big(\mathrm{ToBits}(h_1) \| 0^{\lambda-64}, \lambda\big)$
$h_2 := r_0 h_0 + r_1 h_1'$
$h_3 := r_2 h_0 + r_3 h_1'$
$h := \big(\mathrm{ToBits}(h_2) \| \mathrm{ToBits}(h_3)_{[0..B)}\big) \oplus \mathbf{x}_1$
return h

Algorithm 7: ZK.OWFProve$((\mathbf{w}, \mathbf{u}, \mathbf{V}, ct, \mathrm{iv}_{\mathrm{owf}}, \mathsf{chall}_2))$

for $i \in [0..\ell)$ **do**
  $w[i] \leftarrow \mathsf{ToField}(w[i]; 1)$
for $i \in [0..\ell + \lambda)$ **do**
  $v[i] \leftarrow \mathsf{ToField}(V|_i; \lambda)$
Parse $w[i]_{i\in[0..n)}, v[i]_{i\in[0..n)}$ as $pt.value, pt.mac$
Parse $w[i]_{i\in[n..2n)}, v[i]_{i\in[n..2n)}$ as $x_1.value, x_1.mac$
$(\alpha_0^{dot}, \alpha_1^{dot}) \leftarrow \mathrm{SHAKE}(\mathrm{iv}_{\mathrm{owf}}\|0), \mathrm{SHAKE}(\mathrm{iv}_{\mathrm{owf}}\|1)$
$((\alpha_1, \beta_1, \gamma_1), (\alpha_2, \beta_2, \gamma_2), x_2) \leftarrow \mathsf{LinearTransform}(pt, x_1, \mathrm{iv}_{\mathrm{owf}}, ct)$
for $j \in [1..2]$ **do**
  $(A_{0,j}, A_{1,j}) \leftarrow \mathsf{MulCstrnts}(\alpha_j, \beta_j, \gamma_j)$
$(A_{0,3}, A_{1,3}) \leftarrow \mathsf{DotCstrnt}(\alpha_0^{dot}, x_1)$
$(A_{0,4}, A_{1,4}) \leftarrow \mathsf{DotCstrnt}(\alpha_1^{dot}, x_2)$
$a_0 \leftarrow A_{0,1}\|A_{0,2}\|A_{0,3}\|A_{0,4}$
$a_1 \leftarrow A_{1,1}\|A_{1,2}\|A_{1,3}\|A_{1,4}$
$u^* \leftarrow \sum_{i\in[0..\lambda)} u[\ell + i]\alpha_\lambda^i, \quad v^* \leftarrow \sum_{i\in[0..\lambda)} v[\ell + i]\alpha_\lambda^i$ `// where` $\alpha_\lambda \in \mathbb{F}_{2^\lambda}$
$\tilde{a} \leftarrow \mathsf{ZKHash}(\mathsf{chall}_2, a_1||u^*)$
$\tilde{b} \leftarrow \mathsf{ZKHash}(\mathsf{chall}_2, a_0||v^*)$
return $(\tilde{a}, \tilde{b})$

Algorithm 8: $\mathsf{LinearTransform}(pt, x_1, \mathrm{iv}_{\mathrm{owf}}, ct)$

`// First S-box`
$s_1 := M_1 \cdot pt \oplus \gamma_1$
$y_1 := x_1$
$z_1 := y_1^{2^{e_1}}$
`// Second S-box input`
$t := y_1 \oplus pt$
$s_2 := M_2 \cdot t \oplus \mathrm{iv}_{\mathrm{owf}}$
$y_2 := M_3^{-1} \cdot \big(ct \oplus s_2 \oplus \gamma_2 \oplus pt\big)$
$z_2 := y_2^{2^{e_2}}$
return $\big((s_1, y_1, z_1), (s_2, y_2, z_2), y_2\big)$

Algorithm 9: $\mathsf{MulCstrnts}(x, y, z)$

// x,y,z are represented bitwise
// Each bit wire a has a.value $\in \mathbb{F}_2$ and a.mac $\in \mathbb{F}_{2^\lambda}$
for $i \in [0..n)$ **do**
    $x_i^{(0)} := x_i.\text{mac}$
    $x_i^{(1)} := x_i.\text{mac} \oplus x_i.\text{value} \cdot 1_\lambda$
    $y_i^{(0)} := y_i.\text{mac}$
    $y_i^{(1)} := y_i.\text{mac} \oplus y_i.\text{value} \cdot 1_\lambda$
for $i \in [0..n)$ **do**
    for $j \in [0..n)$ **do**
        $P_{i+j}^{(0)} := P_{i+j}^{(0)} \oplus x_i.\text{mac} \cdot y_j.\text{mac}$
        $P_{i+j}^{(1)} := P_{i+j}^{(1)} \oplus (x_i.\text{mac} \oplus x_i.\text{value}) \cdot (y_j.\text{mac} \oplus y_j.\text{value})$
$P = P \bmod f(X)$
for $k \in [0..n)$ **do**
    $A_{0,k} := P_k^{(0)} \oplus z_k.\text{value}$
    $A_{1,k} := A_{0,k} \oplus P_k^{(1)} \oplus z_k.\text{mac}$
$A_0 := A_{0,0} \| A_{0,1} \| \cdots \| A_{0,n-1}$
$A_1 := A_{1,0} \| A_{1,1} \| \cdots \| A_{1,n-1}$
return (A_0, A_1)

Algorithm 10: $\mathsf{DotCstrnt}(\alpha, x)$

$d.\text{value} \leftarrow 0$
$d.\text{mac} \leftarrow 0$
for $i \in [0..n)$ **do**
    if $\alpha_i = 1$ **then**
        $d.\text{value} \leftarrow d.\text{value} \oplus x_i.\text{value}$
        $d.\text{mac} \leftarrow d.\text{mac} \oplus x_i.\text{mac}$
$A_0 \leftarrow 0$
$A_1 \leftarrow d.\text{mac}$
return (A_0, A_1)

Algorithm 11: $\mathsf{ZKHash}(\mathsf{chall}, (\mathbf{x}_0, x_1))$

// $\mathbf{x}_0 = (x_{0,0}, \ldots, x_{0,m-1}) \in \mathbb{F}_{2^\lambda}^m$
// $x_1 \in \mathbb{F}_{2^\lambda}$
// $\mathsf{chall} \in \{0,1\}^{3\lambda+64}$
$(r_0, r_1, s, t) := \mathsf{chall}$
// $r_0, r_1, s \in \{0,1\}^\lambda, \;\; t \in \{0,1\}^{64}$
$r_0 := \mathrm{ToField}(r_0, \lambda)$
$r_1 := \mathrm{ToField}(r_1, \lambda)$
$s := \mathrm{ToField}(s, \lambda)$
$t := \mathrm{ToField}\big(t \| 0^{\lambda-64}, \lambda\big)$
$h_0 := 0$
for $i \in [0..m)$ **do**
  $h_0 := h_0 + s^{m-1-i} \cdot x_{0,i}$
$h_1 := 0$
for $i \in [0..m)$ **do**
  $h_1 := h_1 + t^{m-1-i} \cdot x_{0,i}$
$h := \mathrm{ToBits}\big(r_0 h_0 + r_1 h_1 + x_1\big)$
return h

Algorithm 12: $\mathsf{BAVC.Open}(\mathrm{decom}, I)$

Parse $\mathrm{decom} = (\mathcal{T}, (\mathrm{leafcom}_{i,j})_{i\in[0..\tau),\, j\in[0..N_i)})$
Parse $\mathcal{T} = (K_0, K_1, \ldots, K_{2L-2})$
$N_{\max} := \max_{i\in[0..\tau)} N_i$
$t := 0$
for $j \in [0..N_{\max})$ **do**
  for $i \in [0..\tau)$ **do**
    if $j < N_i$ **then**
      $\pi(t) := (i, j)$
      $\pi^{-1}(i, j) := t$
      $t := t + 1$
$\mathcal{H} := \emptyset$
for $i \in [0..\tau)$ **do**
  $t_i := \pi^{-1}(i, I_i)$
  $\mathcal{H} := \mathcal{H} \cup \{L - 1 + t_i\}$
$\mathcal{S} := \emptyset$
for $t \in [0..L)$ **do**
  if $L - 1 + t \notin \mathcal{H}$ **then**
    $\mathcal{S} := \mathcal{S} \cup \{L - 1 + t\}$
for $p = L - 2, L - 3, \ldots, 0$ **do**
  if $2p + 1 \in \mathcal{S}$ ***and*** $2p + 2 \in \mathcal{S}$ **then**
    $\mathcal{S} := \big(\mathcal{S} \setminus \{2p + 1, 2p + 2\}\big) \cup \{p\}$
if $|\mathcal{S}| > T_{\mathrm{open}}$ **then**
  return $\perp$
$\mathcal{O} := \emptyset$
foreach $p \in \mathcal{S}$ **do**
  $\mathcal{O} := \mathcal{O} \cup \{(p, K_p)\}$
for $i \in [0..\tau)$ **do**
  $\mathrm{hleaf}_i := \mathrm{leafcom}_{i,I_i}$
$\mathrm{decom}_I := \big(\mathcal{O}, (\mathrm{hleaf}_0, \ldots, \mathrm{hleaf}_{\tau-1})\big)$
return decom_I

2.3.3 Signature Verification.

The verification algorithm of the GreatWall signature is given in algorithm 13. It takes as input the public key pk, the message m and the signature σ and outputs either Accept or Reject.

Phase 1. The verifier first parses the signature into its individual components. It recomputes $\mu := H_1(ct \| m; 2\lambda)$ from the public key and the message.

Before reconstructing the VOLE transcript, the verifier checks whether the grinding condition is satisfied. Specifically, it verifies that the last w_{grind} bits of

chall_3 are all zero:

$$\mathsf{chall}_3[\lambda - w_{\mathsf{grind}} : \lambda) = 0^{w_{\mathsf{grind}}}.$$

If this condition fails, the verifier immediately rejects the signature.

Phase 2. Assuming the grinding check succeeds, the verifier reconstructs the VOLE receiver side from the partial decommitment. It invokes

$$(\mathsf{com}, Q) \leftarrow \mathsf{VOLEReconstruct}\big(\mathsf{decom}_I, \mathsf{chall}_3[0..\lambda - w_{\mathsf{grind}}), c_1, \ldots, c_{\tau-1}, \mathrm{iv}_{\mathrm{vole}}\big).$$

This procedure uses the index information encoded in chall_3 to reconstruct all opened GGM leaves, recompute the corresponding batch commitment com, and recover the verifier's VOLE matrix Q. If the reconstruction algorithm returns $\perp$, the verifier rejects. Once com and Q have been obtained, the verifier recomputes the first Fiat–Shamir challenge by hashing the message digest, the reconstructed commitment, the correction vectors, and $\mathrm{iv}_{\mathrm{vole}}$:

$$\mathsf{chall}_1 := H_2^1\big(\mu \| \mathsf{com} \| c_1 \| \cdots \| c_{\tau-1} \| \mathrm{iv}_{\mathrm{vole}}; 5\lambda + 64\big).$$

Phase 3. The verifier next performs the VOLE consistency check. It first uses a loop to recover the real Q with the correction vectors c_i. Then it compresses the reconstructed VOLE matrix using the universal hash determined by chall_1:

$$\widetilde{Q} := \mathsf{VOLEHash}(\mathsf{chall}_1, Q).$$

Then it parses the bits of chall_3 as

$$\mathsf{chall}_3 = (\delta_0, \ldots, \delta_{\lambda-1}) \in \{0,1\}^\lambda.$$

Using these bits and the signature component $\widetilde{u}$, the verifier removes the challenge-dependent VOLE offset from $\widetilde{Q}$ and obtains

$$h_V := H_1(\widetilde{Q} \oplus [\delta_0 \cdot \widetilde{u} \parallel \delta_1 \cdot \widetilde{u} \parallel \cdots \parallel \delta_{\lambda-1} \cdot \widetilde{u}]).$$

The verifier then recomputes the second Fiat–Shamir challenge as

$$\mathsf{chall}_2 := H_2^2\big(\mathsf{chall}_1 \| \widetilde{u} \| h_V \| d; 3\lambda + 64\big),$$

where D is encoded in column-major order.

Phase 4. Finally, the verifier checks the QuickSilver proof and the consistency of the final Fiat–Shamir challenge. It runs the verification procedure for the OWF proof:

$$\widetilde{b} \leftarrow \mathsf{ZK.OWFVerify}\big(d,\ Q[0..\ell + \lambda),\ \mathsf{chall}_2, \mathsf{chall}_3,\ \widetilde{a},\ ct\big).$$

This step recomputes the missing proof component $\widetilde{b}$ from the witness commitment, the reconstructed VOLE information, and the public OWF instance. The verifier then recomputes $\mathsf{chall}_3' := H_2^3\big(\mathsf{chall}_2 \| \widetilde{a} \| \widetilde{b} \| \mathsf{ctr}; \lambda\big)$. The signature is accepted if and only if $\mathsf{chall}_3' = \mathsf{chall}_3$. Otherwise, the verifier rejects.

Algorithm 13: GreatWall.Verify(m, pk, σ)

Parse $\sigma = ((c_i)_{i\in[1..\tau)}, \tilde{\mathbf{u}}, \mathbf{d}, \tilde{a}, \mathrm{decom}_I, \mathsf{chall}_3, \mathrm{iv}_{\mathrm{vole}}, \mathrm{ctr})$
$\mathrm{iv}_{owf}, ct \leftarrow pk$
$\mu \leftarrow H_1(\mathrm{ct}\|\mathrm{m})$
if $\mathsf{chall}_3[\lambda - w_{grind} : \lambda] \neq 0^{w_{grind}}$ **then**
  return false
`// Reconstruct VOLEs and check commitment`
$(\mathrm{com}, Q') \leftarrow \mathsf{VOLEReconstruct}(\mathrm{decom}_I, \mathsf{chall}_3[0 : \lambda - w_{\mathrm{grind}}], c_1, \ldots, c_{\tau-1}, \mathrm{iv}_{\mathrm{vole}})$
if VOLEReconstruct *returned* $\perp$ **then**
  return false
$\mathsf{chall}_1 \leftarrow H_2^1(\mu\|\mathrm{com}\|c_1\|\cdots\|c_{\tau-1}\|\mathrm{iv}_{\mathrm{vole}}; 5\lambda + 64)$
`// Apply the VOLE correction values and check consistency`
$Q_0 \leftarrow Q'_0$
for $i \in [0..\tau)$ **do**
  $b \leftarrow 0$ if $i < \tau_0$ else 1
  $(\delta_0, \ldots, \delta_{k_b-1}) \leftarrow \mathsf{ChalDec}(\mathsf{chall}_3, i; \mathsf{param})$
  $\widetilde{D}_i \leftarrow [\,\delta_0 \cdot \widetilde{u} \parallel \cdots \parallel \delta_{k_b-1} \cdot \widetilde{u}\,] \in \{0,1\}^{(\lambda+B)\times k_b}$
  if $i > 0$ **then**
    $Q_i \leftarrow Q'_i \oplus [\,\delta_0 \cdot c_i \parallel \cdots \parallel \delta_{k_b-1} \cdot c_i\,]$
$Q \leftarrow [\,Q_0 \ \cdots \ Q_{\tau-1}\,]$
$\widetilde{Q} \leftarrow \mathsf{VOLEHash}(\mathsf{chall}_1, Q) \in \{0,1\}^{(\lambda+B)\times\lambda}$
$h_V \leftarrow H_1\Big(\widetilde{Q} \oplus [\widetilde{D}_0 \ \cdots \ \widetilde{D}_{\tau-1}]\Big)$
$\mathsf{chall}_2 \leftarrow H_2^2(\mathsf{chall}_1\|\tilde{\mathbf{u}}\|h_V\|\mathbf{d}; 3\lambda + 64)$
`// Compute Pylon consistency values`
$\widetilde{b} \leftarrow \mathsf{ZK.OWFVerify}(\mathbf{d}, \mathbf{Q}_{[0..\ell+\lambda)}, \mathsf{chall}_2, \mathsf{chall}_3, \widetilde{a}, ct, \mathrm{iv}_{owf})$
Compute $\mathsf{chall}'_3 \leftarrow H_2^3(\mathsf{chall}_2\|\tilde{a}\|\tilde{b}\|\mathrm{ctr}; \lambda)$
return true if $\mathsf{chall}_3 = \mathsf{chall}'_3$ else false

Algorithm 14: $\mathsf{VOLEReconstruct}(\mathsf{chall}_3, \mathrm{decom}_I, \mathrm{iv}_{\mathrm{vole}}, \widehat{\ell})$

if $\mathsf{chall}_3[\lambda - w_{\mathrm{grind}}..\lambda) \neq 0^{w_{\mathrm{grind}}}$ **then**
  return $\perp$
$\mathrm{pos} := 0$
for $i \in [0..\tau)$ **do**
  $\kappa_i := \begin{cases} k_0, & i < \tau_0, \\ k_1, & i \geq \tau_0 \end{cases}$
  $N_i := 2^{\kappa_i}$
  $I_i := \sum_{s=0}^{\kappa_i - 1} \mathsf{chall}_3[\mathrm{pos} + s] \cdot 2^s$
  $\mathrm{pos} := \mathrm{pos} + \kappa_i$
if $\mathrm{pos} \neq \lambda - w_{\mathrm{grind}}$ **then**
  return $\perp$
Parse $\mathrm{decom}_I = (\mathcal{O}, (\mathrm{hleaf}_0, \ldots, \mathrm{hleaf}_{\tau-1}))$
$L := \sum_{i=0}^{\tau-1} N_i$
$N_{\max} := \max_{i \in [0..\tau)} N_i$
$t := 0$
for $j \in [0..N_{\max})$ **do**
  for $i \in [0..\tau)$ **do**
    if $j < N_i$ **then**
      $\pi(t) := (i, j)$
      $\pi^{-1}(i, j) := t$
      $t := t + 1$
for $p \in [0..2L - 1)$ **do**
  $K_p := \perp$
foreach $(p, K) \in \mathcal{O}$ **do**
  $K_p := K$
for $p \in [0..L - 1)$ **do**
  if $K_p \neq \perp$ **then**
    $(K_{2p+1}, K_{2p+2}) := \mathsf{PRG}(K_p, \mathrm{iv}_{\mathrm{vole}}; 2\lambda)$
for $t \in [0..L)$ **do**
  $(i, j) := \pi(t)$
  if $j = I_i$ **then**
    $\mathrm{sd}_{i,j} := \perp$
    $\mathrm{leafcom}_{i,j} := \mathrm{hleaf}_i$
  else
    if $K_{L-1+t} = \perp$ **then**
      return $\perp$
    $(\mathrm{sd}_{i,j}, \mathrm{leafcom}_{i,j}) := H_0(K_{L-1+t} \| \mathrm{iv}_{\mathrm{vole}})$
for $i \in [0..\tau)$ **do**
  $h_i := H_1(\mathrm{leafcom}_{i,0} \| \cdots \| \mathrm{leafcom}_{i,N_i-1})$
$\mathrm{hcom} := H_1(h_0 \| h_1 \| \cdots \| h_{\tau-1})$
for $i \in [0..\tau)$ **do**
  $\Delta_i := I_i$
  $\mathrm{sd}'_{i,0} := \perp$
  for $j \in [1..N_i)$ **do**
    $\mathrm{sd}'_{i,j} := \mathrm{sd}_{i,j \oplus \Delta_i}$
  $(u_i^0, q_{i,0}, \ldots, q_{i,\kappa_i-1}) := \mathsf{ConvertToVOLE}(\mathrm{sd}'_{i,0}, \ldots, \mathrm{sd}'_{i,N_i-1}, \mathrm{iv}_{\mathrm{vole}}, \widehat{\ell})$
  $Q_i^0 := [q_{i,0} \ \cdots \ q_{i,\kappa_i-1}] \in \mathbb{F}_2^{\widehat{\ell} \times \kappa_i}$
return $(\mathrm{hcom}, Q_0^0, \ldots, Q_{\tau-1}^0)$

Algorithm 15: ZK.OWFVerify$((\mathbf{d}, \mathbf{Q}, \mathsf{chall}_2, \mathsf{chall}_3, \widetilde{a}, ct, \mathrm{iv}_{\mathrm{owf}}))$

$\Delta := \mathrm{ToField}(chall_3)$
Write $Q := [\,Q_0 \ \cdots \ Q_{\tau-1}\,]$ // $Q_0, \ldots, Q_{\tau_0-1}$ have k_0 columns, others k_1
for $i \in [0..\tau)$ **do**
  $b \leftarrow 0$ if $i < \tau_0$ else 1
  $(\delta_0, \ldots, \delta_{k_b-1}) \leftarrow \mathsf{ChalDec}(\mathsf{chall}_3, i; \mathsf{param})$
  $Q_i \leftarrow [\,q_{i,0}, \ldots, q_{i,k_b-1}\,]$ // consider k_b columns separately
  for $j \in [0, k_b)$ **do**
    $q_{i,j}[0..\ell-1] \leftarrow q_{i,j}[0..\ell-1] \oplus \delta_j \cdot d$ // perform $q[j] := q[j] + d[j] \cdot \Delta$ column-wise
for $i \in [0..\ell+\lambda)$ **do**
  $q[i] \leftarrow \mathrm{ToField}(Q[i]; \lambda)$ // create VOLE keys row-wise (transpose Q)
$b \leftarrow 0$;
$((\alpha_1, \beta_1, \gamma_1), (\alpha_2, \beta_2, \gamma_2), q_{x_2}) \leftarrow \mathsf{LinearTransform}(q_{pt}, q_{x_1}, \mathrm{iv}_{\mathrm{owf}}, ct)$
$dot_0 = \alpha_0^{dot} * \beta_1 \ dot_1 = \alpha_1^{dot} * \beta_2$
$b \leftarrow (\alpha_1.mac * \beta_1.mac - \gamma_1.mac * \Delta, \alpha_2.mac * \beta_2.mac - \gamma_2.mac * \Delta, dot_0 * \Delta + \Delta^2, dot_1 * \Delta + \Delta^2)$;
$q^* \leftarrow \sum_{i \in [0..\lambda)} q[\ell+i]\alpha_\lambda^i$ // $\alpha_\lambda \in \mathbb{F}_{2^\lambda}$
$\widetilde{q} \leftarrow \mathsf{ZKHash}(\mathsf{chall}_2, (b \| q^*))$
return $\widetilde{q} - \widetilde{a} \cdot \Delta$

2.3.4 Additional Functions.

Our construction relies on the same symmetric primitives as FAEST v1. Specifically, we use the following primitives:

- $\mathsf{PRG} : \{0,1\}^\lambda \times \{0,1\}^{128} \to \{0,1\}^*$, a pseudorandom generator taking as input a λ-bit seed and a 128-bit initialization vector.
- $H_0 : \{0,1\}^{\lambda+128} \to \{0,1\}^\lambda \times \{0,1\}^{2\lambda}$, a hash function for commitment generation.
- $H_1 : \{0,1\}^* \to \{0,1\}^{2\lambda}$, a collision-resistant hash function.
- $H_2^j : \{0,1\}^* \to \{0,1\}^*$, for $j \in \{1,2,3\}$, hash functions used to derive the three Fiat–Shamir challenges and modeled as random oracles.
- $H_3 : \{0,1\}^* \to \{0,1\}^{\lambda+128}$, a hash function used to derive the signature randomness and initialization vector.

The pseudorandom generator PRG is instantiated using AES-λ in CTR mode with seed s and initialization vector $\mathrm{iv}_{\mathrm{vole}}$. We write $\mathsf{PRG}(s, \mathrm{iv}_{\mathrm{vole}}; \ell)$ to denote the first ℓ output bits of the keystream.

All hash functions are instantiated identically to FAEST v1, using SHAKE128 for $\lambda = 128$ and SHAKE256 otherwise. To achieve domain separation, a one-byte domain identifier is appended to the input before hashing. For variable-length outputs, we write $H_i(x; \ell)$ to denote an ℓ-bit hash output.

Algorithm 16: $\mathsf{ToField}(x, k)$

// α_k is a primitive element of $\mathbb{F}_{2^k}$
if $x \in \{0,1\}^k$ **then**
  return $\sum_{i=0}^{k-1} x[i]\alpha_k^i$
if $x \in \{0,1\}^{nk}$ **then**
  for $i \in [0..n)$ **do**
    $y[i] := \sum_{j=0}^{k-1} x[ki+j]\alpha_k^j$
  return $y \in \mathbb{F}_{2^k}^n$
return $\perp$

Algorithm 17: $\mathsf{ToBits}(x, k, n)$

$y := \epsilon$
for $i \in [0..n)$ **do**
  Parse $x[i] = x_0 + x_1\alpha_k + \cdots + x_{k-1}\alpha_k^{k-1},\ x_j \in \mathbb{F}_2$
  $y := y \| (x_0 \cdots x_{k-1})$
return $y \in \{0,1\}^{nk}$

Algorithm 18: $\mathsf{ChalDec}(\mathsf{chall}_3, i; \mathsf{param})$

Input: $\mathsf{chall}_3 \in \{0,1\}^\lambda$, index $i \in [0..\tau)$, parameters param
Output: $(\delta_0, \ldots, \delta_{k_i-1})$

$n_{\mathsf{ch}} \leftarrow \lambda - w_{\mathsf{grind}}$
$k_0 \leftarrow \lceil n_{\mathsf{ch}}/\tau \rceil$
$k_1 \leftarrow \lfloor n_{\mathsf{ch}}/\tau \rfloor$
$\tau_0 \leftarrow n_{\mathsf{ch}} \bmod \tau$
$\tau_1 \leftarrow \tau - \tau_0$ // $\tau_0 k_0 + \tau_1 k_1 = n_{\mathsf{ch}}$
if $i < \tau_0$ **then**
  $k_i \leftarrow k_0$
  $\mathsf{off}_i \leftarrow i \cdot k_0$
else
  $k_i \leftarrow k_1$
  $\mathsf{off}_i \leftarrow \tau_0 k_0 + (i - \tau_0)k_1$
$(\delta_0, \ldots, \delta_{k_i-1}) \leftarrow \mathsf{chall}_3[\mathsf{off}_i : \mathsf{off}_i + k_i]$
return $(\delta_0, \ldots, \delta_{k_i-1})$

Algorithm 19: DecodeAllChall3(chall_3; param)

for $i \in [0..\tau)$ **do**
  $(\delta_0, \ldots, \delta_{k_i-1}) \leftarrow \mathsf{ChalDec}(\mathsf{chall}_3, i; \mathsf{param})$
  $I_i \leftarrow \sum_{j=0}^{k_i-1} \delta_j 2^j$ `// same bits as ChalDec`
return $(I_0, \ldots, I_{\tau-1})$

3 Security Goals

This section states the security goals of GreatWall and clarifies the scope of the security claims. The purpose of this section is not to provide the full proof, but to specify what is claimed by the scheme. The reductions are given in Sect. 6.6, and the dedicated cryptanalytic evaluation of Pylon is given in Sect. 6.

3.1 Correctness

GreatWall is required to satisfy correctness for honestly generated keys and honestly generated signatures. For a key pair (pt, ct) generated by the key-generation algorithm, where $ct = \mathsf{Pylon}(\mathrm{iv}_{\mathrm{owf}}, pt)$, any signature σ honestly generated on a message m should be accepted by the verification algorithm, namely $\mathsf{Verify}(ct, m, \sigma) = 1$. The signing algorithm may repeat the challenge generation when the grinding condition is not satisfied or when the batch all-but-one vector commitment opening aborts. These events only affect the expected signing time. Once the signing algorithm outputs a signature, the verifier deterministically checks the transcript and accepts an honestly generated signature except with negligible probability coming from the underlying randomized procedures.

3.2 Unforgeability

The primary security goal of GreatWall is existential unforgeability under adaptive chosen-message attacks. An adversary is given the public key and may adaptively request signatures on messages of its choice. The adversary wins if it outputs a valid signature on a new message that was not previously queried to the signing oracle.

GreatWall aims to achieve this security notion in both the classical random oracle model and the quantum random oracle model. In the classical setting, the adversary is a probabilistic polynomial-time algorithm with classical access to the random oracles. In the post-quantum setting, the adversary may be a quantum polynomial-time algorithm with quantum access to the random oracles, while signing queries remain classical.

At a high level, under the simulation and extraction conditions of the proof system, a successful forgery against GreatWall must either break the soundness of the VOLE-in-the-Head proof system or imply an algorithm for finding a

preimage of the public Pylon output. The corresponding formal reductions are provided in Sect. 6.6.

3.3 One-Wayness of Pylon

The primitive-specific security goal of GreatWall is the preimage resistance of Pylon. Given a public key (iv_{owf}, ct) generated by the actual KeyGen algorithm including the rejection conditions, it should be computationally infeasible to find any pt' such that $\mathrm{Pylon}(iv_{\mathrm{owf}}, pt') = ct$. This is the primitive-specific assumption required by the conditional ROM reduction in Sect. 6.6.3 and Sect. 6.6.4.

This goal is analyzed from two perspectives. First, Sect. 6.6.2 examines the preimage experiment in the ideal-permutation model, where the nonlinear layers are modeled as independent public random permutations. It identifies the remaining requirements for a quantitative bound. Second, Sect. 6 evaluates the concrete Pylon instances against attacks that exploit their actual algebraic structure. The dedicated analysis includes brute-force preimage search in Sect. 6.1, algebraic attacks in Sect. 6.2, statistical attacks in Sect. 6.3, and quantum attacks in Sect. 6.4, with the overall conclusion summarized in Sect. 6.5.

Thus,the conditional reduction connects signature forgery to preimage recovery under its stated assumptions. The dedicated attack analysis provides evidence relevant to those assumptions.

3.4 Scope of the Security Claims

The security claims of GreatWall apply to the algorithms and parameter sets specified in this document. The construction does not rely on number-theoretic, lattice-based, code-based, or multivariate public-key assumptions. Instead, its security is based on the preimage resistance of Pylon, the soundness and zero-knowledge properties of the VOLEitH proof system, and the random-oracle modeling of the hash functions used in the Fiat–Shamir transform.

The claims in this document are cryptographic security claims at the algorithmic level. Physical attacks, side-channel attacks, fault attacks, randomness failures, and attacks caused by incorrect or non-constant-time implementations are outside the scope of the formal security analysis and require separate implementation-level countermeasures.

GreatWall is specified at four security levels. Three variants—GreatWall-192, GreatWall-256, and GreatWall-512—correspond directly to the mandatory security levels of the NGCC solicitation. GreatWall-128 is provided as an additional variant for applications that do not require quantum security above 64 bits. Table 3 lists the correspondence between GreatWall variants and classical and quantum security strengths.

GreatWall's security rests on two independent pillars: the one-wayness of the Pylon one-way function and the knowledge soundness of the underlying VOLEitH proof system. An adversary who succeeds in forging a signature must either find a preimage of Pylon, or break the knowledge-soundness of the zero-knowledge argument; a conditional ROM reduction from signature forgery to

Table 3: GreatWall variants and security levels.

Variant	n	NGCC Level	Classical security (bits)	Quantum security (bits)
GreatWall-128	137	–	128	68
GreatWall-160	163	Level 1	160	80
GreatWall-192	197	Level 1	192	96
GreatWall-256	263	Level 2	256	128
GreatWall-512	521	Level 3	512	256

preimage recovery for concrete Pylon under the actual KeyGen distribution is given in Theorem 1. No structured algebraic hardness assumption is introduced at any point in the construction.

For the intended security targets stated in Table 3, implementations must ensure that each key pair is used with a fresh randomness seed per signing invocation and that the public key is transmitted and stored with integrity.

4 Features

4.1 A Modular Signature Design Centered on Pylon

GreatWall is organized as a modular signature scheme centered on the one-way function Pylon. The complete signature scheme, denoted by GreatWall, consists of the key-generation, signing, and verification algorithms, together with the VOLE-in-the-Head proof system, Fiat–Shamir challenges, GGM-tree-based batch all-but-one vector commitment, and the concrete parameter choices for each security level. In contrast, Pylon is the underlying one-way function that defines the public-key relation to be proven. The secret key is a preimage pt, and the public key is the corresponding output $(\mathrm{iv}_{\mathrm{owf}}, ct)$ satisfying $ct = \mathsf{Pylon}(\mathrm{iv}_{\mathrm{owf}}, pt)$.

This separation makes the role of each component explicit. The VOLEitH layer provides the generic proof-and-signature mechanism: it turns a proof of knowledge of a valid preimage into a publicly verifiable signature. Pylon, on the other hand, determines the concrete algebraic relation, the witness representation, and the main cost of the proof. Thus, GreatWall should be viewed as the full signature construction, while Pylon should be viewed as the dedicated one-way-function instance embedded inside that construction.

This modular structure is useful both for specification and for security analysis. The generic security of the signature scheme is analyzed through the soundness and zero-knowledge properties of the VOLEitH proof system, together with the Fiat–Shamir transform in the random oracle model and quantum random oracle model. The concrete security of the public-key relation is analyzed separately through the preimage resistance of Pylon. As a result, the overall security claim of GreatWall is decomposed into a proof-system part and a primitive-specific part, which makes the assumptions and the scope of each analysis clear.

4.2 Proof-Cost-Aware One-Way Function Design

A distinguishing feature of GreatWall is that the underlying one-way function is designed with the cost structure of VOLEitH proofs in mind. In a VOLE-based proof, linear operations can be processed almost for free by exploiting the linearity of the VOLE correlations, whereas nonlinear multiplication constraints dominate the proof size, proving time, and verification cost. Therefore, the main design objective for Pylon is to keep the number of nonlinear constraints small without making the public relation algebraically simple.

This objective is achieved by using inverse Mersenne mappings as the only nonlinear components. For an inverse Mersenne S-box, the relation between its input x and output y can be checked by $xy = y^{2^e}$. The map $y \mapsto y^{2^e}$ is a Frobenius map and is therefore $\mathbb{F}_2$-linear over $\mathbb{F}_{2^n}$. Hence the above relation contains only one genuine nonlinear operation, namely the multiplication xy. As a result, each S-box layer contributes only one field multiplication constraint to the QuickSilver-style consistency check.

This property has a direct impact on the proof relation used in GreatWall. The verifier does not need a separate nonlinear check for every linear layer, constant addition, or feed-forward addition. These parts are absorbed into linear transformations of the authenticated witness values. Consequently, the nonlinearity of the whole Pylon relation is concentrated in two multiplication checks, corresponding to the two inverse Mersenne layers.

This proof-cost-aware design is important for the overall efficiency of GreatWall. It reduces the extended witness length and the number of multiplication constraints, which are the dominant terms in VOLEitH-based signatures. At the same time, the low proof cost is not obtained by using a trivially invertible relation: the dense linear layers and nested feed-forward structure are included to preserve algebraic coupling between the two nonlinear layers. Thus, Pylon is optimized not merely as a standalone primitive, but as a one-way function whose algebraic form is suitable for efficient zero-knowledge proof generation.

4.3 Low-Round Structure with Security-Oriented Coupling

Pylon adopts a two-round structure in order to reduce the cost of proving the underlying one-way-function relation. In GreatWall, the size of the extended witness is directly affected by the number of full-state nonlinear layers. For an r-round construction of this form, the extended witness length is $\ell_r = rn$. Thus, replacing a three-round relation by a two-round relation removes one full n-bit nonlinear state from the witness. Since the witness is authenticated across all VOLE repetitions, this reduction is amplified in the final signature size.

However, the design does not rely on a low round number alone. A naive two-round construction may allow the internal relation to be separated into simpler one-round components, or may permit the surrounding linear layers to be merged and eliminated by algebraic manipulation. Pylon therefore combines the two-round structure with dense linear layers and nested feed-forward connections.

These components are introduced to preserve strong interaction between the two nonlinear layers even though the number of rounds is small.

The outer feed-forward branch binds the secret input to the final public output and prevents the two S-box layers from being treated as independent local transformations. The inner feed-forward branches inject earlier state information into later nonlinear computations, which increases the coupling among the input, intermediate values, and output. As a result, the public relation remains compact for the prover, but it is not reduced to a simple cascade of independently invertible layers.

This feature is essential for the design philosophy of GreatWall. The two-round architecture provides the main efficiency gain, while the security-oriented coupling mechanisms are used to compensate for the reduced round count. In this sense, Pylon is not merely a round-reduced primitive; it is a proof-efficient two-round one-way function whose internal dependencies are deliberately arranged to resist algebraic decomposition.

4.4 Parameter Selection Against Known Algebraic Shortcuts

The parameter choices of Pylon are driven not only by efficiency, but also by resistance against known algebraic shortcuts on power mappings over binary extension fields. In particular, the extension degrees are chosen to be prime, and the inverse Mersenne exponents are selected after checking the subgroup-linearization conditions that appear in recent attacks on round-reduced symmetric primitives.

This is important because power mappings over $\mathbb{F}_{2^n}$ may admit decompositions when $2^n - 1$ has suitable nontrivial divisors. In such cases, part of the nonlinear layer can sometimes be transformed into an $\mathbb{F}_2$-linear Frobenius map after guessing a subgroup value. This may substantially reduce the effective number of variables and lead to guess-and-linearize or overdefined quadratic attacks. Pylon avoids this attack path by choosing prime extension degrees and by excluding exponents that admit such decompositions.

For the recommended instances, the exponents are screened so that the inverse Mersenne mappings cannot be rewritten in the form

$$x^{\bar{e}} = (x^d)^s \cdot x^{2^t}$$

for a useful nontrivial divisor $d \mid 2^n - 1$. Hence the attacker cannot turn an S-box layer into a linear residual map by a low-cost subgroup guess. The same screening is applied to both nonlinear layers, and the exponent pairs are chosen so that no common divisor enables simultaneous linearization of the two S-boxes.

This parameter-selection strategy is a feature of the construction rather than a post-hoc check. It ensures that the efficiency gained from using binary-field Frobenius structure does not introduce the subgroup structures required by known linearization-based attacks. The concrete divisor checks and dedicated algebraic analysis are given later in the security evaluation, while this section highlights the design principle behind the selected parameters.

4.5 Scalable Security Levels with a Uniform Architecture

GreatWall is designed to scale across different security levels without changing the overall structure of the signature scheme or the underlying one-way function. The scheme provides four parameter sets, namely GreatWall-128, GreatWall-192, GreatWall-256, and GreatWall-512. These instances use the same Pylon architecture, the same VOLEitH proof framework, and the same signing and verification procedures. Across security levels, the main changes are the field size, the inverse Mersenne exponents, and the VOLEitH parameters required to meet the corresponding security target.

This uniformity is useful for both implementation and analysis. From the implementation perspective, the same code structure can be reused across all four security levels, with only parameter-dependent field arithmetic and proof parameters being adjusted. From the analysis perspective, the same security arguments apply to all instances: the generic signature security is derived from the VOLEitH proof system and the Fiat–Shamir transform, while the concrete one-wayness analysis is instantiated with the corresponding Pylon parameters.

The scalability of Pylon mainly comes from enlarging the binary extension field. Increasing n directly increases the preimage-search space and the cost of generic algebraic attacks, while preserving the same two-round architecture and the same type of S-box relation. Hence higher-security instances do not require adding more rounds, introducing additional parallel branches, or replacing the primitive with a different construction.

This feature is particularly important for long-term post-quantum deployment. GreatWall provides a consistent path from 128-bit security to conservative long-term security levels, while keeping the specification, implementation framework, and security analysis uniform across the family.

4.6 Compact-Key and Signature-Size-Oriented Performance

GreatWall is designed with compact keys and signature-size-oriented performance in mind. Since the public key is one output of the Pylon one-way function and the secret key is the corresponding preimage, the key sizes remain small across all parameter sets. This feature is useful in applications where public keys, certificates, or authentication data need to be transmitted, stored, or verified frequently.

The dominant communication cost of GreatWall comes from the VOLEitH proof rather than from the public-key material. Therefore, the performance-oriented part of the design focuses on reducing the size of the relation that must be proven. Pylon contributes to this goal in two ways. First, its two-round structure reduces the extended witness length by removing one full nonlinear state compared with a three-round construction. Second, its inverse Mersenne S-boxes keep the number of multiplication constraints small, so that the QuickSilver-style consistency check remains compact.

This gives GreatWall a clear performance profile within the family of symmetric-key-based and VOLEitH-based signatures. The scheme combines compact public

keys with a proof relation specifically optimized for short VOLEitH signatures. The resulting construction is intended to provide a practical balance between signature size, signing time, and verification time on standard 64-bit platforms.

Overall, the performance advantage of GreatWall comes from the co-design of the signature framework and the underlying one-way function. The VOLEitH layer supplies a modular proof-and-signature mechanism, while Pylon reduces the cost of the proven relation. This combination allows GreatWall to obtain shorter signatures among comparable VOLEitH-based designs while preserving a simple and scalable construction, without relying on number-theoretic, lattice-based, code-based, or multivariate public-key assumptions.

5 Design Rationale

5.1 Choice of a Two-Round Architecture.

The number of rounds of the underlying one-way function affects the VOLEitH signature size through the extended witness length ℓ. In our setting, each full-state nonlinear layer contributes one state of n authenticated bits to the proof relation, while the surrounding linear layers, constants, and feed-forward operations are handled locally by the linearity of VOLE. Hence, for an r-round construction of this form, the extended witness length is $\ell_r = rn$. This observation motivates minimizing the number of nonlinear rounds. A three-round construction gives $\ell_3 = 3n$, whereas a two-round construction gives $\ell_2 = 2n$. Therefore, moving from three rounds to two rounds reduces the extended witness length by$\Delta\ell = \ell_3 - \ell_2 = n$. Thus this translates into an absolute signature-size saving of

$$\Delta|\sigma| = \tau n \tag{4}$$

bits for all VOLEitH-based instantiations considered in this paper.

However, achieving security at two rounds is far from automatic. The recent break of two-round Rain [43] demonstrates that naively removing nonlinear rounds may substantially weaken the resulting algebraic system. Instead, Pylon addresses this problem by combining two full-state inverse Mersenne S-box layers with dense linear layers and a *nested feed-forward structure*. The dense linear layers spread each nonlinear transformation over the entire state, while the feed-forward terms make the two nonlinear layers couple more tightly and prevent a direct separation into two independent one-round problems.

Together, these mechanisms are intended to compensate for the reduced number of rounds and to preserve sufficient algebraic complexity in the resulting one-way function. And concrete signature-size reductions obtained from this choice are reported in Sect. 7.

5.2 Choice of Field.

For a symmetric primitive intended for VOLEitH-style zero-knowledge protocols, the choice of the underlying field must balance efficiency in secure evaluation

and resistance against algebraic cryptanalysis. Binary extension fields $\mathbb{F}_{2^n}$ are naturally attractive in this setting, since Frobenius maps $x \mapsto x^{2^t}$ are $\mathbb{F}_2$-linear and can be implemented very efficiently. However, this same structure may also introduce additional algebraic regularity that can be exploited by linearization-based attacks. In particular, recent attacks on round-reduced Rain and full AIM-III [43] show that, when $2^n - 1$ admits suitable nontrivial divisors, a power mapping can sometimes be rewritten in the form $x^k = (x^d)^s \cdot x^{2^t}$, so that after guessing the subgroup value x^d, the remaining part becomes an $\mathbb{F}_2$-linear map and the nonlinear layer can be partially reduced to a linear or low-degree system.

To avoid this structural weakness, we choose the extension degree n to be a prime number. This choice is motivated by security rather than implementation convenience. When n is prime, the factorization structure of $2^n - 1$ is much more restrictive than in the even-degree case, and in particular it excludes the special decompositions exploited in the known linearization attacks on inverse- and Mersenne-type power maps. For our target application in VOLEitH, this is especially desirable: the primitive is used as a low-multiplicative-complexity one-way function inside a VOLE-in-the-head framework, where any algebraic shortcut directly threatens the soundness margin of the proof system. By working over $\mathbb{F}_{2^n}$ with prime extension degree, we retain the efficiency benefits of binary-field arithmetic while deliberately avoiding extension degrees that admit the subgroup structures required by such attacks. Therefore, the choice of field in our design is guided not only by implementation efficiency, but also by the need to suppress linearization-based algebraic attacks in the VOLEitH setting.

We note that the chosen prime degrees n are only slightly larger than the security parameter λ, giving an overhead at most 7% in field-element size and less than 3% for $\lambda \geq 192$. This marginal gap has a negligible impact on arithmetic cost. The extended witness length increases from 2λ to $2n$ bits, contributing fewer than 18 additional authenticated bits per repetition at the 128-bit level, which is absorbed into the constant terms of the signature-size formula. In practice, the benchmarks in Section 7 confirm that this overhead is not a bottleneck.

5.3 Choice of Nonlinear Layer: Inverse Mersenne Mapping.

In the VOLEitH setting, the proof cost is closely tied to the nonlinear part of the underlying relation. Linear operations can usually be handled very efficiently, whereas each nonlinear multiplication contributes directly to the online proving cost and also affects the overall proof size and verification complexity. Therefore, the nonlinear layer should satisfy two requirements simultaneously: it should be economical to represent inside the VOLEitH protocol, and it should avoid algebraic structures that admit overly compact equation systems. To achieve this goal, we instantiate the nonlinear layer by the *inverse Mersenne mapping*

$$S(x) = x^{\bar{e}}, \qquad \bar{e} = (2^e - 1)^{-1} \pmod{2^n - 1}, \tag{5}$$

over $\mathbb{F}_{2^n}$, where $\gcd(e, n) = 1$ so that S is a permutation. This choice is motivated by the following considerations.

First, the inverse Mersenne mapping provides a flexible family of power permutations with high algebraic degree, while being more tunable than the fixed inverse mapping $x^{-1} = x^{2^n-2}$. In a VOLEitH protocol, the nonlinear layer is not used in isolation, but as part of an efficiently provable one-way-function relation. Hence, it is desirable to have a family of exponents that allows the proof cost and the algebraic properties of the primitive to be optimized jointly. The inverse Mersenne form offers exactly this flexibility: by varying the parameter e, one can search for exponents that retain strong algebraic complexity while leading to more efficient circuit representations than plain inversion.

Second, this choice is attractive in the VOLEitH setting because it admits a simple algebraic verification rule. Let

$$y = x^{\bar{e}}, \qquad \bar{e} = (2^e - 1)^{-1} \pmod{2^n - 1}. \tag{6}$$

Then one equivalently has

$$y^{2^e-1} = x, \qquad \text{or equivalently} \qquad y \cdot x = y^{2^e}. \tag{7}$$

Hence, the S-box can be verified through a single multiplication constraint, which is particularly efficient in VOLEitH. This gives the inverse Mersenne mapping a favorable feature as a nonlinear layer with both low proof cost and strong algebraic structure.

Third, from the perspective of algebraic modeling, this choice also offers an important security advantage. A standard inverse S-box admits relatively many implicit quadratic relations between its input and output, and in particular it is known to yield up to $5n$ linearly independent quadratic equations. By contrast, Mersenne-type power mappings yield only $3n$ such equations. Therefore, using an inverse Mersenne mapping helps limit the number of low-degree relations available to the attacker, making it harder to derive an overdefined quadratic system [28]. This is particularly desirable in the VOLEitH setting: while the proof system benefits from low multiplicative complexity, the primitive should not at the same time expose too many quadratic constraints that could simplify algebraic cryptanalysis.

5.3.1 Choice of the Exponents.

The exponents e_1 and e_2 are selected according to both VOLEitH efficiency and algebraic-security requirements.

On the efficiency side, we require that the corresponding inverse exponents

$$\bar{e}_i = (2^{e_i} - 1)^{-1} \pmod{2^n - 1} \tag{8}$$

admit efficient arithmetic realizations and lead to low nonlinear cost inside the VOLEitH proof system. Since the proof cost is dominated by nonlinear operations, the exponents must be chosen so that the resulting S-boxes remain practical to evaluate while still providing sufficiently strong algebraic mixing.

In particular, we favor exponents whose inverse Mersenne forms can be implemented with short multiplication chains, so that both prover computation and multiplication verification remain efficient.

On the algebraic side, we require $\gcd(e_i, n) = 1$ so that $x \mapsto x^{2^{e_i}-1}$, and hence also its inverse $x \mapsto x^{\bar{e}_i}$, is a permutation over $\mathbb{F}_{2^n}$. Moreover, the exponents are chosen so that the induced inverse Mersenne mappings have high algebraic degree and ensure rapid degree growth across rounds, which is important for reaching full-state algebraic diffusion with only a small number of rounds. At the same time, we avoid exponents that would introduce excessive low-degree structure or unusually sparse quadratic descriptions.

Most importantly, the inverse Mersenne exponents $\bar{e}_1, \bar{e}_2$ are selected to avoid known linearization-based algebraic attacks. Concretely, for our parameter sets there is no nontrivial divisor $d \mid (2^n - 1)$ such that $\bar{e}_i \equiv 2^t \pmod{d}$ for some integer t and for either $i \in \{1, 2\}$. Hence the nonlinear mapping $S_i(x) = x^{\bar{e}_i}$ cannot be rewritten in the form $x^{\bar{e}_i} = (x^d)^s \cdot x^{2^t}$ with a nontrivial subgroup component and an $\mathbb{F}_2$-linear residual term. In addition, the pair (e_1, e_2) is chosen so that no common nontrivial divisor $d \mid (2^n - 1)$ simultaneously enables such a decomposition for both nonlinear layers. Therefore, the simultaneous linearization mechanism underlying known attacks on power mappings over $\mathbb{F}_{2^n}$ does not apply to our inverse Mersenne S-boxes [43].

5.3.2 Linear Layer: Random Dense Matrix.

For the linear layer, we use multiplication by a randomly chosen dense matrix $M \in \mathbb{F}_2^{n \times n}$. The matrices are selected so that their linearized-polynomial representations are dense and have max degree of 2^{n-1} in $\mathbb{F}_{2^n}$. This unstructured affine layer serves two critical purposes. First, it ensures full diffusion at the bit level. Second, when combined with the nonlinear operations and feed-forward structure, it rapidly increases the degree of regularity of the algebraic system after only a few rounds, making algebraic attacks computationally infeasible.

5.3.3 Nested Feed-forward Structure.

Pylon employs three forward-feedback connections to enhance cross-round interaction and to obstruct algebraic decomposition. Instead of letting each round be viewed as an almost independent composition of one linear layer and one S-box, the feedback paths repeatedly re-inject earlier state information into later stages, so that intermediate variables become tightly fused throughout the entire permutation. This is especially desirable in our design, since recent decomposition-based algebraic attacks on arithmetic-oriented primitives [38] explicitly exploit structural simplifications of the internal relation, either by splitting it into simpler low-degree components or by reducing several linear layers to a single equivalent one. The forward-feedback structure is therefore introduced to strengthen the coupling among intermediate variables and to obstruct both types of simplification. A quantitative assessment of this security contribution is given in Sect. 6.2, where we analyze two stripped variants of Pylon with feed-forward

connections partially or fully removed: without any feed-forward the function becomes directly invertible at cost $O(n^\omega)$, and retaining only the outer feed-forward reduces the attack complexity to approximately $2^{n-1+e_1+e_2}$, far below the $2^{\omega(n-1+e_1+e_2)}$ cost of the full construction. This confirms that both layers of feed-forward are necessary for the two-round design to achieve the target security level.

6 Security Evaluations

6.1 Brute-force Attack

A straightforward way to attack Pylon is exhaustive search over the whole preimage space. Given a public instance $(\mathrm{iv}_{\mathrm{owf}}, ct)$, where $ct = \mathrm{Pylon}(\mathrm{iv}_{\mathrm{owf}}, pt)$ for an unknown secret input pt, an adversary may simply enumerate all 2^n possible candidates $pt \in \mathbb{F}_{2^n}$, evaluate $\mathrm{Pylon}(\mathrm{iv}_{\mathrm{owf}}, pt)$, and check whether the result equals ct. Therefore, the time complexity of the brute-force preimage attack is $O(2^n)$, with average complexity about 2^{n-1}, while the memory is negligible.

6.2 Algebraic Attacks

In this part, we analyze algebraic attacks on Pylon. It must be emphasized that Pylon is used as a one-way function within the VOLEitH framework. To be specific, the adversary is restricted to a known pair $(\mathrm{iv}_{\mathrm{owf}}, ct)$, aimed at recovering the secret input pt, s.t., $\mathrm{Pylon}(\mathrm{iv}_{\mathrm{owf}}, pt) = ct$.

Gröbner Basis Attacks. Our primary aim is to model Pylon as a system of algebraic equations. To solve for the unknown variables, we use Gröbner basis algorithms [15]. By performing an order transformation on the computed basis, the multivariate system can be converted into a triangular form containing univariate equations, which are then solvable. The system of equations can be modeled by several ways.

Equation System over $\mathbb{F}_2$. We analyze Gröbner basis attacks by modeling Pylon as a Boolean polynomial system over $\mathbb{F}_2$. We introduce intermediate state variables between rounds so that each round contributes a block of quadratic constraints induced by the nonlinear layer. With r rounds, this gives a quadratic system with rn Boolean variables and vrn quadratic equations, together with the standard Boolean field equations $x^2 - x = 0$ for all variables. Under the semi-regular assumption [2], the corresponding Hilbert series can be written as $\mathrm{HS}(z) = (1+z)^{rn}(1-z^2)^{vrn}$. We estimate the degree of regularity d_{reg} as the smallest degree d at which the coefficient of z^d in $\mathrm{HS}(z)$ becomes non-positive. Then the complexity of computing a Gröbner basis can be estimated by the standard linear-algebra bound $O\left(\binom{rn+d_{\mathrm{reg}}}{d_{\mathrm{reg}}}^{\omega}\right)$, where ω is usually set to 2. In our concrete parameters, we set $r = 2$ and $v = 3$ since the inverse Mersenne mapping provides $3n$ linearly independent quadratic equations over $\mathbb{F}_2$ on the n state bits, hence

$$\mathrm{HS}(z) = (1+z)^{2n}(1-z^2)^{6n}. \tag{9}$$

The resulting d_{reg} estimates and the corresponding bit complexities are summarized in Table 4.

Scheme	n	v	d_{reg}	Bit complexity
Pylon-128	137	3	15	163.7
Pylon-160	163	3	17	188.5
Pylon-192	197	3	20	224.2
Pylon-256	263	3	25	286.3
Pylon-512	521	3	45	532.5

Table 4: Estimated degree of regularity and complexity of Gröbner-basis attacks over $\mathbb{F}_2$ for Pylon.

Experimental Validation on Reduced Instances. We performed concrete Gröbner-basis computations on reduced Pylon instances over $\mathbb{F}_2$. For each reduced dimension n, we generated random inputs $x \in \mathbb{F}_{2^n}$, computed $y = \mathrm{Pylon}(\mathrm{iv}_{\mathrm{owf}}, x)$, and encoded the inversion problem $\mathrm{Pylon}(\mathrm{iv}_{\mathrm{owf}}, X) = y$ as a Boolean polynomial system. We used the same intermediate-state formulation as in the complexity estimate, with $r = 2$ rounds and $v = 3$ quadratic constraints per state bit.

We stress that this formulation is favorable to the attacker, since the use of intermediate variables keeps the system quadratic and avoids the high-degree expansion of the composed Pylon map. The Gröbner bases were computed in grevlex order using Magma [12]. We recorded the solving degree sd, defined as the highest degree reached during the Gröbner-basis computation.

The results are shown in Table 5. For the solved instances ($n \leq 19$), the observed solving degree matches the Hilbert-series estimate ($sd = d_{\mathrm{reg}}$ or $d_{\mathrm{reg}} - 1$), consistent with the semi-regularity assumption [2]. The running time grows rapidly with n: instances with $n \geq 23$ did not terminate within the time limit.

Extended Degree-Limited Experiments. To further probe the behavior at larger dimensions, we ran an additional set of degree-limited Gröbner-basis computations for $n \in \{23, 29, 31, 37\}$ using Magma's `DegreeLimit` parameter set to d_{reg}. In all four cases, the computation reached step degree 4 but did not advance to the predicted $d_{\mathrm{reg}} \in \{5, 6\}$ within the 300s wall-clock limit (Table 6). Peak memory consumption (Peak RSS) grew from 1.34 GB at $n = 23$ to 10.33 GB at $n = 37$, indicating exponential growth of intermediate basis elements as the computation approaches d_{reg}. This is consistent with semi-regular system behavior

Crucially, *no anomalous low-degree termination was observed in any instance.* The basis computation progressed normally through degree 4 without early collapse or unexpected degree drop. Early termination below the predicted d_{reg} would be the signature of an exploitable low-degree structure; its absence across all tested dimensions confirms that Pylon does not exhibit such weakness. The inability to reach d_{reg} within the resource limit is consistent with the exponential intermediate-basis growth characteristic of semi-regular systems, rather than any structural shortcut.

n	r	v	#Var.	#Quad. Eq.	d_{reg}	sd	Median time	Result
11	2	3	22	66	4	4	0.10 s	5/5 solved
13	2	3	26	78	4	4	0.90 s	5/5 solved
17	2	3	34	102	4	4	21.37 s	5/5 solved
19	2	3	38	114	5	4	98.06 s	5/5 solved
23	2	3	46	138	5	–	> 3 h	timeout
29	2	3	58	174	6	–	> 3 h	timeout

Table 5: Experimental Gröbner-basis attacks on reduced Pylon instances over $\mathbb{F}_2$. For $n \leq 19$, we generated five random instances and report the median solving time. For $n \in \{23, 29\}$, the first random instance did not terminate within the time limit. The sd denotes the highest degree reached during the grevlex Gröbner-basis computation.

n	d_{reg}	`DegreeLimit`	Step degree reached	Peak RSS	Result
23	5	5	4	1.34 GB	timeout (> 300 s)
29	6	6	4	5.36 GB	timeout (> 300 s)
31	6	6	4	5.11 GB	timeout (> 300 s)
37	6	6	4	10.33 GB	timeout (> 300 s)

Table 6: Degree-limited Gröbner-basis experiments for larger reduced Pylon instances. `DegreeLimit` is set to d_{reg} in each case. The step degree reached before timeout is 4 in all instances, with no anomalous low-degree termination.

Equation System over $\mathbb{F}_{2^n}$. Another approach is to construct the equation systems over $\mathbb{F}_{2^n}$. We can model the function as a univariate polynomial equation. However, due to the high algebraic degree of the inverse Mersenne S-box and the dense linear layer, the resulting polynomial reaches nearly the maximal degree of $(2^n - 2)$ after a single round. Root-finding algorithms for such high-degree dense polynomials require a complexity of approximately $\mathcal{O}(2^{3n})$ [20], rendering this approach computationally intractable. Consequently, we focus our security analysis on multivariate equation systems over $\mathbb{F}_{2^n}$. Considering the fact that $y = x^{\bar{e}} \implies x = y^{2^e-1}$, we can add intermediate variables to decrease the degree of the inverse Mersenne S-box. By introducing an intermediate variable x_i in each round i after S-box, we obtain equation systems in the following:

$$\begin{cases} M_1(pt) + \gamma_1 = x_1^{2^{e_1}-1}, \\ M_2(x_1 + pt) + \text{iv}_{\text{owf}} = x_2^{2^{e_2}-1}, \\ M_3(x_2) + x_2^{2^{e_2}-1} + \gamma_2 + pt = ct. \end{cases} \tag{10}$$

Here we denote the S-box output in round i by x_i. Note that the three equations have a high degree, since M_i is a linearized polynomial of maximum high degree of 2^{n-1} in our construction. Hence the degree of regularity [1] in this system can be assumed to be $D_{reg} = 1 + \sum_{i=1}^{3} \deg(f_i) - 1 \geq 2^{n-1}$, hence we can see that

$$\binom{r + 1 + 2^{n-1}}{2^{n-1}}^{\omega} > \binom{1 + 2^{n-1}}{2^{n-1}}^{\omega} = (1 + 2^{n-1})^{\omega} > 2^n, \tag{11}$$

namely $\log_2 \left(\binom{r+1+2^{n-1}}{2^{n-1}}^{\omega} \right) > n$. So it is infeasible for Gröbner Basis attacks.

Overdefined Quadratic Modeling Attacks. A recent line of work demonstrated that 3-round Rain can be modeled as a highly overdefined multivariate quadratic polynomial system by exploiting the field inversion at the boundary rounds [28]. More specifically, their construction uses identities that are special to $y = x^{-1}$, following the idea of Courtois–Pieprzyk [14].

Let x_1 denote the output of the first inversion layer and x_2 the input to the last inversion layer in 3-round Rain, and let $\vec{x}_1, \vec{x}_2 \in \mathbb{F}_2^\ell$ be the coordinate (bit) vectors of $x_1, x_2 \in \mathbb{F}_{2^\ell}$ with respect to a fixed basis of $\mathbb{F}_{2^\ell}$. This allows them to derive an extra 5ℓ quadratic Boolean equations that involve only $(\vec{x}_1, \vec{x}_2)$, which makes the resulting system significantly more overdefined. This mechanism is closely linked to the algebra of the inverse function. Besides the basic relation $xy = 1$, one can derive up to $5n$ linearly independent quadratic Boolean relations from $y = x^{-1}$ when it is expanded over $\mathbb{F}_2$. Our design deliberately replaces the inversion with the inverse Mersenne S-box $S(x) = x^{\bar{e}}$. In contrast to the inverse map, the inverse Mersenne S-box admits only about $3n$ linearly independent quadratic Boolean relations of the type used in quadratic modeling, as analyzed in AIM2 [24]. Therefore, the specific overdefinition boost exploited for 3-round Rain does not directly carry over to our setting, and the attacker cannot obtain the same amount of free quadratic constraints from the S-box layer alone. In addition, our surrounding linear layers and the feed-forward structure further hinder the boundary elimination tricks that are essential in [28]. Taken together, we do not expect the multivariate quadratic system from our construction to become highly overdefined, which is required for this type of attack. Therefore, we consider this attack ineffective for our chosen parameters.

Resultant Attacks. Resultant-based algebraic attacks have recently attracted increasing attention in the cryptanalysis of symmetric primitives. In particular, such attacks have been successfully applied to several arithmetic (AO) primitives, including Rescue-Prime [39,3], JARVIS [39], AIM [38], AIM2 [33] and so on. These results demonstrate that even primitives designed for algebraic proof systems can still have algebraic weaknesses when their internal structure produces systems with usable resultants. In this part, we analyze the resistance of Pylon against resultant attacks. We follow the general attack framework proposed in prior attacks on AIM [38] and AIM2 [33].

MITM Modeling for Pylon. We apply a meet-in-the-middle (MITM) modeling strategy to the algebraic system induced by our construction [38]. Specifically, we introduce M_a and subsequently introduce the inverse matrices M_a^{-1} in order to cancel the effect. Let $L_1(x), L_2(x), L_3(x)$ and $L_a(x)$ denote the linearized polynomials associated with the matrices M_1, M_2, M_3 and M_a, respectively. Denote $L_\alpha(x) = L_a(L_1(x)), L_\beta(x) = L_a(L_2^{-1}(x)), L_\gamma(x) = L_a(L_3(x))$, and the output of the first two S-box as v_1 and v_2. This modeling derives a system of bivariate equations as

$$\begin{cases} L_\alpha(pt) = L_a(v_1^{2^{e_1}-1}), \\ L_a(v_1 + pt) = L_\beta(v_2^{2^{e_2}-1}), \\ L_\gamma(v_2) = L_a(pt + ct + v_2^{2^{e_2}-1}). \end{cases} \tag{12}$$

To eliminate the variable v_1 and v_2, we rewrite the equation as

$$\begin{cases} g_1 := L_\alpha(pt) + L_a(v_1^{2^{e_1}-1}) = 0, \\ g_2 := L_a(v_1 + pt) + L_\beta(v_2^{2^{e_2}-1}) = 0, \\ g_3 := L_\gamma(v_2) + L_a(pt + ct + v_2^{2^{e_2}-1}) = 0. \end{cases} \tag{13}$$

Constructing Low-degree Equations. Based on the linearized polynomial decomposition method [38], let

$$L_1(x) = \sum_{j=0}^{n-1} d_{1,j} x^{2^j}, L_2^{-1}(x) = \sum_{j=0}^{n-1} d_{2,j} x^{2^j}, L_3(x) = \sum_{j=0}^{n-1} d_{3,j} x^{2^j}, L_a(x) = \sum_{i=0}^{n-1} a_i x^{2^i},$$

where $d_{1,j}$'s, $d_{2,j}$'s, $d_{3,j}$'s are known coefficients while a_i's are to be determined. Assume that

$$\deg(L_a(x)) \le 2^l, \deg(L_\alpha(x)) \le 2^\gamma, \deg(L_\beta(x)) \le 2^\beta, \deg(L_\gamma(x)) \le 2^\tau,$$

then we can get a system of $(3n-\gamma-\beta-\tau-3)$ equations with $(l+1)$ unknowns:

$$\begin{cases} a_i = 0 \quad \text{for } l+1 \le i \le n-1, \\ \sum_{\substack{0 \le i,j \le n-1 \\ i+j \equiv t_1 \bmod n}} a_i d_{1,j}^{2^i} = 0 \quad \text{for } \gamma+1 \le t_1 \le n-1, \\ \sum_{\substack{0 \le i,j \le n-1 \\ i+j \equiv t_2 \bmod n}} a_i d_{2,j}^{2^i} = 0 \quad \text{for } \beta+1 \le t_2 \le n-1, \\ \sum_{\substack{0 \le i,j \le n-1 \\ i+j \equiv t_3 \bmod n}} a_i d_{3,j}^{2^i} = 0 \quad \text{for } \tau+1 \le t_3 \le n-1. \end{cases} \tag{14}$$

For a nontrivial solution to exist, the variable count should exceed the equation count, i.e., $l + \gamma + \beta + \tau + 4 > 3n$.

Extracting a Univariate Equation. The goal is to eliminate the variables v_1 and v_2 to obtain a univariate polynomial of pt. Note that the substitution theory [39] is not applicable here. Firstly we compute $G_1(pt, v_2) = R(g_1, g_2, v_1)$ to eliminate v_1. The size of the Sylvester matrix is at most $(2^l+2^l\cdot(2^{e_1}-1))\times(2^l+2^l\cdot(2^{e_1}-1))$, and after this the degree of pt and v_2 are bounded by $D_1 := \deg_{pt}(G_1) \le 2^l \cdot 2^l \cdot (2^{e_1}-1) + 2^\gamma \cdot 2^l$ and $\deg_{v_2}(G_1) \le 2^\beta \cdot (2^{e_2}-1) \cdot 2^l \cdot (2^{e_1}-1) \approx 2^{\beta+l+e_1+e_2}$. Secondly we compute $G_2(pt) = R(G_1, g_3, v_2)$ to eliminate v_2. The degree of v_2 in g_3 is $D_2 := \deg_{v_2}(g_3) = \max(2^\tau, (2^{e_2}-1)\cdot 2^l)$ The Sylvester matrix has size of most $(2^{\beta+l+e_1+e_2} + D_2) \times (2^{\beta+l+e_1+e_2} + D_2)$, and after this the degree of pt is bounded by

$$d := \deg_{pt}(G_2) \le D_1 \cdot D_2 + 2^l \cdot 2^{\beta+l+e_1+e_2}. \tag{15}$$

Finally we solve $G_2(pt)$ and get the solution.

Complexity Analysis. The main computational cost stems from the resultant elimination of v_2 and solving $G_2(pt)$. The complexity of eliminating v_2 [4] can be estimated as

$$\mathcal{O}(D_1(\log D_1)^2(2^{\beta+l+e_1+e_2} + D_2)^\omega), \tag{16}$$

and the complexity of solving $G_2(pt)$ [5,35] can be estimated as

$$\mathcal{O}(d \log d(\log d + \log q \log \log d)). \tag{17}$$

Limitation of the MITM resultant approach. Although the above MITM–resultant strategy is well-defined, it does not lead to a competitive attack on Pylon. Unlike the case of AIM, the intermediate variables in Pylon cannot be merged into a single variable together with a useful substitution structure. As a result, one has to eliminate v_1 and v_2 sequentially. However, the first elimination already causes a significant blow-up in the degree with respect to the remaining variable v_2, namely $\deg_{v_2}(G_1)$ becomes very large. Consequently, the second resultant dominates the overall cost, and the resulting complexity is higher than that of direct algebraic modeling. Hence, we do not regard the above resultant-based approach as the best attack on Pylon, and turn to direct modeling.

Direct Modeling. Let $L_1(x), L_2(x)$ and $L_3(x)$ denote the linearized polynomials associated with the matrices M_1, M_2 and M_3, respectively. Denote the output of the second S-box as v. Then we have

$$\begin{cases} L_1(pt) + \gamma_1 = (L_2^{-1}(v^{2^{e_2}-1} + \mathrm{iv}_{\mathrm{owf}}) + pt)^{2^{e_1}-1}, \\ L_3(v) + v^{2^{e_2}-1} + \gamma_2 + pt = ct. \end{cases} \tag{18}$$

To eliminate the variable v, we rewrite the equation as

$$\begin{cases} g_1 := L_1(pt) + \gamma_1 + (L_2^{-1}(v^{2^{e_2}-1} + \mathrm{iv}_{\mathrm{owf}}) + pt)^{2^{e_1}-1} = 0, \\ g_2 := L_3(v) + v^{2^{e_2}-1} + \gamma_2 + pt + ct = 0. \end{cases} \tag{19}$$

We compute $G(pt) = R(g_1, g_2, v)$ to eliminate the variable v. The Sylvester matrix has size of most $(2^{n-1}\cdot(2^{e_2}-1)\cdot(2^{e_1}-1)+2^{n-1})\times(2^{n-1}\cdot(2^{e_2}-1)\cdot(2^{e_1}-1)+2^{n-1})$, and after this we have $d := \deg_{pt}(G) \le 2^{n-1}\cdot(2^{e_2}-1)\cdot(2^{e_1}-1)+2^{n-1}\cdot 2^{n-1}$. Following the complexity formula in [38] introduced above, the dominant cost is

$$\log_2 T \approx \underbrace{(n-1) + 2\log_2(n-1)}_{\log_2[D_2(\log D_2)^2]} + \underbrace{\omega\,(n-1+e_1+e_2)}_{\omega \log_2 D_1}, \tag{20}$$

where $D_1 = 2^{n-1}(2^{e_1}-1)(2^{e_2}-1)$ and $D_2 = 2^{n-1}$ are the degrees of g_1 and g_2 in v, respectively. We note that D_1 already exceeds the brute-force cost 2^n for all recommended parameter sets, so the resultant computation alone is more expensive than exhaustive search. In our parameter setting, the estimated complexity of the above direct modeling resultant attack is summarized in Table 7, which shows the security of Pylon against resultant attack.

Role of the Feed-Forward Structure. To quantify the security contribution of the nested feed-forward structure, we analyze two stripped variants of Pylon.

Case 1: No feed-forward. Without any feed-forward connection, M_1 and M_3 can be eliminated by direct substitution, reducing the system to

$$\begin{cases} g_1 := pt + t_1^{2^{e_1}-1} = 0, \\ g_2 := M_2(t_1) + \mathrm{iv}_{\mathrm{owf}} + ct^{2^{e_2}-1} = 0. \end{cases} \tag{21}$$

where $\mathsf{ct}^{2^{e_2}-1}$ is a known constant. Since g_2 is linear in t_1, the unique solution $t_1 = M_2^{-1}(\mathsf{ct}^{2^{e_2}-1} - \mathrm{iv}_{\mathrm{owf}})$ is recovered by a single matrix inversion, and $\mathsf{pt} = t_1^{2^{e_1}-1}$ follows immediately. The total attack cost is $O(n^\omega)$ field operations, which is at most 2^{26} for all recommended parameter sets—the function is directly invertible and provides no one-wayness guarantee.

Case 2: Outer feed-forward only. Retaining the outer feed-forward while removing the two inner connections, the relation $\mathsf{pt} = \mathsf{ct} + M_3(v) + \gamma_2$ allows pt to be substituted out, and $M_1 \circ M_3$ is absorbed into a single redefined linear map $\tilde{M} := M_1 \circ M_3$, so that the first and third linear layers introduce no additional degree. The system becomes

$$\begin{cases} g_1 := t_1^{2^{e_1}-1} + \tilde{M}(v) + c_1 = 0, \\ g_2 := M_2(t_1) + \mathrm{iv}_{\mathrm{owf}} + v^{2^{e_2}-1} = 0. \end{cases} \tag{22}$$

where c_1 is a known constant. Again g_2 is linear in t_1, so t_1 is eliminated by direct substitution, yielding a univariate polynomial in v of degree $2^{n-1}(2^{e_1}-1)(2^{e_2}-1)$. The attack cost is dominated by solving this polynomial, with $\log_2 T \approx n - 1 + e_1 + e_2$, giving approximately 2^{281} for Pylon-128—above the security target but far below the 2^{937} cost of the full construction.

Taken together, these two cases show that the inner feed-forward connections are responsible for preventing the linear elimination of t_1 from g_2: in the full Pylon, the inner paths inject pt into the input of the second S-box, raising $\deg_{\mathsf{pt}}(g_1)$ from 1 to $2^{e_1} - 1$ and forcing a full resultant computation whose cost grows as $2^{\omega(n-1+e_1+e_2)}$. The outer feed-forward in turn prevents round decomposition by binding the two S-box layers to a single global input-output relation. Both layers of feed-forward are therefore necessary for the two-round design to achieve its target security level.

Scheme	n	Time complexity
Pylon-128	137	937.0
Pylon-160	163	1092.3
Pylon-192	197	1317.2
Pylon-256	263	1753.7
Pylon-512	521	3452.8

Table 7: Complexities of resultant attack on Pylon ($\omega = 2.8$) [38,33].

Algebraic Attacks Using Linearization Technique. The algebraic attack of Zhang et al. [43] on round-reduced Rain is based on a guess-and-linearize strategy that exploits the power-map structure of the large inversion S-box. For one-round Rain, the attacker guesses a suitable value $x^d = \alpha$, where $d \mid (2^n - 1)$, and rewrites the inversion in the form $x^{-1} = (x^d)^s \cdot x^{2^t}$. After guessing α, the term $(x^d)^s$ becomes a constant, while x^{2^t} is an $\mathbb{F}_2$-linear Frobenius map. As a result, the nonlinear S-box is transformed into a linear expression, and the resulting

system can be solved efficiently by Gaussian elimination. For two-round Rain, the same idea is not applied independently to both S-boxes, since linearizing each inversion layer already incurs a cost of about $(2^n-1)/d$, which would eliminate the gain over exhaustive search. Instead, the first S-box is linearized to reduce the number of effective variables, and the second S-box is then exploited to derive an overdefined quadratic system in the remaining variables. In this setting, [43] choose divisors of the special form $d = 2^w + 1$, which arise naturally from the factorization of $2^n - 1$ for even n. After guessing $(P + k + c^{(1)})^d = \alpha$, the first S-box becomes linear, and the relation

$$(P + k + c^{(1)})^{2^{2w}} = \alpha^{2^w-1}(P + k + c^{(1)}) \tag{23}$$

yields additional linear equations. Consequently, the internal state can be expressed using only $2w$ free variables. The second S-box is then represented as quadratic equations in these free variables, and the resulting system is finally solved by standard linearization and Gaussian elimination.

Our parameter choices are specifically intended to rule out the above guess-and-linearize strategy. First, unlike the parameter sets targeted in [43], all our extension degrees n are chosen as odd primes. Hence, the special factorization pattern of 2^n-1 used in the Rain attack for even n does not arise in our setting. In particular, the decomposition underlying the linearization of the inverse map, which relies on divisors of the form $d = 2^w + 1$ derived from the even-degree structure of $\mathbb{F}_{2^n}$, is no longer available. Therefore, the first step of the attack, namely reducing the inversion layer to an $\mathbb{F}_2$-linear expression by guessing a subgroup value $x^d = \alpha$, cannot be carried out in the same way. Second, for our two Mersenne-type exponents e_1 and e_2, the parameters are selected so as to avoid any nontrivial divisor $d \mid (2^n - 1)$ satisfying $(2^{e_i} - 1)^{-1} \equiv 2^t \pmod d$ for some integer t and for either $i \in \{1, 2\}$. Consequently, the S-boxes cannot be rewritten in the form $x^{\overline{e_i}} = (x^d)^s \cdot x^{2^t}$ with a nontrivial subgroup guess and an $\mathbb{F}_2$-linear residual term. Moreover, there is no common nontrivial divisor $d \mid (2^n-1)$ that would simultaneously enable such a decomposition for both exponents e_1 and e_2. This rules out the simultaneous linearization mechanism that is essential in the attack on full AIM-III in [43]. Hence, for our parameter sets, the algebraic attack of [43] loses its crucial entry point: the adversary can no longer exploit a low-cost subgroup guess to transform one nonlinear layer into a linear system and then derive a sufficiently overdefined quadratic system from the next layer. Therefore, our instances are resistant to the specific linearization-based subgroup attacks developed in [43].

For completeness, we explicitly verify the subgroup-linearization condition for all recommended parameter sets. For an exponent e, define

$$\mathcal{D}_e(n) = \left\{ d > 1 : d \mid (2^n - 1),\ \exists t \in \{0, \ldots, n-1\} \text{ s.t. } (2^e - 1)^{-1} \equiv 2^t \pmod d \right\}. \tag{24}$$

Here the inverse is taken modulo d. Since our parameters satisfy $\gcd(e, n) = 1$, we have

$$\gcd(2^e - 1, 2^n - 1) = 2^{\gcd(e,n)} - 1 = 1, \tag{25}$$

and hence 2^e-1 is invertible modulo every divisor of 2^n-1. If a composite divisor d satisfies the above condition, then every prime divisor $p \mid d$ also satisfies $(2^e-1)^{-1} \equiv 2^t \pmod{p}$. Therefore, it suffices to check the condition over all prime divisors of 2^n-1. In our verification, we used the complete prime factorizations of 2^n-1 for the recommended values of n. The script first verifies that the product of the listed prime factors is exactly 2^n-1, and then checks each prime divisor p and each $t \in \{0,\dots,n-1\}$. No nontrivial prime-divisor hit was found. By the prime-divisor reduction above, this implies $\mathcal{D}_{e_i}(n)=\emptyset$ for both $i=1,2$ and for all recommended parameter sets.

We also check the divisor pattern used in the linearization attack on reduced RAIN, namely divisors of the form 2^w+1 of 2^n-1. For our choices of n, this condition is ruled out structurally. Indeed, if $2^w+1 \mid 2^n-1$, then $2^w \equiv -1 \pmod{2^w+1}$, so the multiplicative order of 2 modulo this divisor is even. On the other hand, since the divisor also divides 2^n-1, this order must divide the odd prime n, which is impossible. Thus no divisor of the form 2^w+1 occurs for our parameter sets. As shown in Table 8, none of our recommended parameter sets admits a nontrivial subgroup divisor enabling the decomposition $x^{\bar{e}_i} = (x^d)^s \cdot x^{2^t}$ with an $\mathbb{F}_2$-linear residual Frobenius term. Consequently, the guess-and-linearize step required by the attacks of [43] is not available for Pylon.

Variant	n	(e_1, e_2)	$\exists w:\ 2^w+1 \mid 2^n-1$	$\mathcal{D}_{e_1}(n)$	$\mathcal{D}_{e_2}(n)$
Pylon-128	137	$(70, 75)$	none	$\emptyset$	$\emptyset$
Pylon-160	163	$(79, 86)$	none	$\emptyset$	$\emptyset$
Pylon-192	197	$(94, 105)$	none	$\emptyset$	$\emptyset$
Pylon-256	263	$(129, 136)$	none	$\emptyset$	$\emptyset$
Pylon-512	521	$(248, 273)$	none	$\emptyset$	$\emptyset$

Table 8: Parameter checks against subgroup-linearization attacks. The fourth column checks if a divisor of the form 2^w+1 divides 2^n-1, The sets $\mathcal{D}_{e_i}(n)$ contain nontrivial divisors $d \mid 2^n-1$ satisfying $(2^{e_i}-1)^{-1} \equiv 2^t \pmod{d}$ for some $t \in \{0,\dots,n-1\}$.

6.3 Statistical Attacks

In the considered one-wayness setting, the adversary is given a target pair $(\mathrm{iv}_{\mathrm{owf}}, ct)$ generated from an unknown secret input pt. The goal is to find some preimage pt' such that $\mathrm{Pylon}(\mathrm{iv}_{\mathrm{owf}}, pt') = ct$. This is different from the usual block-cipher cryptanalysis setting, where an adversary may obtain many input-output pairs correlated with the same secret key or secret instance. Although Pylon is public and can be evaluated offline on arbitrary inputs, such evaluations do not by themselves provide many samples correlated with the unknown target preimage. Therefore, classical statistical distinguishers would have to be converted into preimage-finding algorithms in order to be useful in our setting. We are not aware of such a conversion that improves over generic preimage search for Pylon.

Differential cryptanalysis. Differential cryptanalysis relies on observing how input differences propagate through many correlated input-output pairs. In our setting, the attacker is given only one target value and aims to recover a preimage. The attacker can of course evaluate the public function on arbitrary inputs, but these evaluations do not give difference pairs involving the unknown preimage pt. Hence, a standard differential attack does not directly translate into a preimage-recovery attack against Pylon in the single-target one-wayness model.

Linear cryptanalysis. Linear cryptanalysis requires estimating biases of linear approximations from many correlated input-output samples. In the one-wayness setting considered here, only one target input-output relation involving the hidden preimage is available. Offline evaluations of the public primitive may reveal global statistical properties of the function, but they do not directly reveal whether the hidden preimage satisfies a useful linear approximation. Therefore, standard linear cryptanalysis does not give a direct advantage over generic preimage search unless such a distinguisher can be turned into an efficient inversion algorithm.

Integral attacks. Integral attacks exploit balancedness or cancellation properties over carefully chosen structured sets of inputs. Such attacks are naturally formulated in settings where the adversary can observe the propagation of an entire structured set through the same secret instance. In our attack model, the adversary only has one target value associated with the unknown preimage. Although the public function can be evaluated on chosen structured sets, this does not directly test whether the hidden preimage belongs to a useful integral structure. Hence, standard integral attacks are not directly applicable in their usual form.

Saturation attacks. Saturation attacks are closely related to integral attacks and similarly rely on tracking how structured sets of inputs propagate through the primitive. Since the one-wayness experiment provides only a single target output and no structured set of correlated evaluations involving the secret preimage, saturation attacks also do not directly yield a preimage-finding method in our model.

Consequently, differential, linear, integral, and saturation attacks do not directly apply in their standard forms to the single-target one-way-function setting considered in this work. In particular, these attacks require many correlated input-output samples, whereas the adversary in our model is given only a single target value associated with the unknown preimage. Offline evaluations of the public function do not provide such correlated samples involving the hidden preimage, and hence do not yield an effective preimage recovery strategy.

6.4 Quantum Attacks

There are two types of quantum attacks. In the first case, an adversary is allowed to use quantum computation but not allowed to make any quantum query, while in the second case both are accessible [42].

Quantum Algebraic Attack. Since each inverse Mersenne S-box contributes $3n$ linearly independent quadratic equations like the S-box in AIM [25] and AIM2 [26], quantum algebraic attacks such as GröverXL algorithm [10] and Quantum-BooleanSolve algorithm [18] yield a complexity of approximately $O(2^{(0.3687+o(1))\cdot 3n})$ and $O(2^{0.462\cdot 3n})$ respectively, which is significantly higher than the generic Grover search complexity of $O(2^{0.5n})$ [23]. Hence we conclude that under the estimates above, the quantum algebraic attacks considered here do not improve upon the generic Grover search[23].

Quantum Generic Attack. Advanced quantum generic attacks, particularly those based on Simon's period-finding algorithm [34] or its offline variants [11], rely on querying a keyed oracle multiple times to detect hidden periods. However, in our setting, the adversary is restricted to a single public target pair $(\mathrm{iv}_{\mathrm{owf}}, ct)$. As the adversary lacks access to a keyed oracle for performing multiple classical or superposition queries, constructing the periodic function required for Simon's algorithm [34] is impossible. Therefore, structural quantum attacks are inapplicable, and the generic security of our primitive is still determined by Grover's algorithm [23], providing $n/2$ bits of security.

6.5 Summary of Dedicated Attack Analysis

Table 9 summarizes the dedicated attacks considered above in the single-target one-way-function setting. Attacks that inherently require many input-output pairs are marked as not applicable.

Attack type	Estimate / status	Conclusion
Brute-force preimage search	2^n	Generic classical baseline
Gröbner basis over $\mathbb{F}_2$	Table 4	Above target
Gröbner basis over $\mathbb{F}_{2^n}$	$> 2^n$	Not competitive
Overdefined quadratic modeling	No RAIN-type overdefinition	Ineffective
MITM resultant modeling	Degree blow-up	Not competitive
Direct resultant attack	Table 7	Above target
Subgroup linearization	Table 8	Ruled out by parameters
Differential / linear cryptanalysis	N/A	Require many correlated samples
Integral / saturation attacks	N/A	Require structured input sets
Quantum algebraic attacks	Worse than Grover	Not best known
Generic quantum preimage search	$2^{n/2}$	Quantum baseline

Table 9: Summary of dedicated attacks on Pylon. "N/A" means that the attack does not apply in the single-target one-way-function setting.

Under the estimates presented above, none of the dedicated attacks evaluated here falls below the stated classical security targets for the recommended parameters. The next subsection presents an ideal-permutation analysis, a conditional ROM reduction, and the scope of the QROM framework.

6.6 Formal Security Analysis

6.6.1 Security Models and Scope of the Generic Analysis

Before presenting the security analysis, we distinguish the idealized model used for generic attacks from the concrete construction and from the assumptions required by the reduction.

Two complementary analyses. The security evaluation of Pylon considers two complementary perspectives:

- *Structural cryptanalysis (Sect. 6).* We evaluate attacks exploiting the inverse Mersenne S-boxes, the linear layers, and the nested feed-forward structure. These analyses concern the concrete Pylon construction and the specific attack classes considered.
- *Generic black-box analysis.* Regard S_1 and S_2 as independent uniformly random permutations and consider adversaries accessing these permutations and their inverses through oracles. This is a separate idealized model; conclusions established in this model do not automatically apply to the concrete algebraic permutations.

Scope of the conclusions. The absence of a faster attack among the methods evaluated in Sect. 6 does not establish a lower bound for every algorithm attacking concrete Pylon. Likewise, an ideal-permutation result does not prove the one-wayness of the concrete inverse-Mersenne construction.

The EUF-CMA reduction in Sect. 6.6.3 therefore retains the preimage resistance of concrete Pylon as an explicit assumption, using the public-key distribution generated by the actual KeyGen algorithm. Its simulation and extraction requirements must also be verified for the complete GreatWall instantiation.

6.6.2 Generic Analysis of Pylon under the Ideal-Permutation Model

Model. Assume that S_1 and S_2 are independent uniformly random permutations of $\{0,1\}^n$. The public maps M_1, M_2, M_3 are invertible $\mathbb{F}_2$-linear maps, and the public constants are fixed.

The challenger may implement the permutations by lazy sampling. A new forward or inverse query is answered consistently with all previously assigned permutation pairs, including assignments made internally when generating the challenge. A fresh response is uniform over the unused values conditional on the full internal permutation table. This statement does not imply independence conditional only on the adversary-visible transcript and the public challenge.

Game 1 (Unconditioned ideal-permutation preimage experiment)**.** *The experiment* $\mathsf{Exp}^{\mathrm{pre,IP}}_{\mathrm{Pylon}}(\mathcal{A})$ *proceeds as follows:*

1. *The challenger independently samples*

$$S_1, S_2 \xleftarrow{\$} \mathrm{Perm}(\{0,1\}^n), \qquad iv_{\mathrm{owf}} \xleftarrow{\$} \{0,1\}^n, \qquad \mathsf{pt} \xleftarrow{\$} \{0,1\}^n.$$

It computes $\mathsf{ct} := \mathsf{Pylon}(iv_{\mathrm{owf}}, \mathsf{pt})x$ *using these permutations and gives* $(iv_{\mathrm{owf}}, \mathsf{ct})$ *to* $\mathcal{A}$*.*

2. *The adversary makes at most q adaptive queries in total to $S_1, S_1^{-1}, S_2, S_2^{-1}$.*
3. *The adversary outputs* pt'.

The adversary wins if $\mathsf{Pylon}(iv_{\mathrm{owf}}, \mathsf{pt}') = \mathsf{ct}$. *And the advantage is defined as*

$$\mathsf{Adv}^{\mathrm{pre,IP}}_{\mathrm{Pylon}}(\mathcal{A}) := \Pr[\mathcal{A} \textit{ wins}].$$

Difference from the signature key distribution. Game 1 uses unconditioned sampling. The actual KeyGen algorithm rejects candidates that do not satisfy its two dot-product conditions. Consequently, this experiment is not the preimage experiment used directly by the signature reduction. Transferring a bound between these distributions requires a separate analysis of the rejection condition and its acceptance probability.

Augmented adversary and transcripts. Let $\mathcal{A}^+$ run $\mathcal{A}$ and, before returning its candidate pt', query the complete evaluation path of that candidate. This adds at most two permutation queries and leaves its output and success probability unchanged.

Let $Q := q + 2$ be an upper bound on the number of queries made by $\mathcal{A}^+$. Write $\mathcal{T}_1 = \{(a_i, y_i)\}, \mathcal{T}_2 = \{(c_j, z_j)\}$ for the distinct input-output records exposed by its queries to the first and second permutations, respectively. Then $|\mathcal{T}_1| + |\mathcal{T}_2| \le Q$. Each record satisfies $S_1(a_i) = y_i$ or $S_2(c_j) = z_j$, respectively.

Consistent pairs. The transcript contains a consistent pair for ct if there exist $(a, y) \in \mathcal{T}_1$ and $(c, z) \in \mathcal{T}_2$ such that

$$c = M_2\big(y \oplus M_1^{-1}(a \oplus \gamma_1)\big) \oplus \mathrm{iv}_{\mathrm{owf}}, \tag{26}$$

$$M_3(z) \oplus c \oplus M_1^{-1}(a \oplus \gamma_1) \oplus \gamma_2 = \mathsf{ct}. \tag{27}$$

Such a pair determines the preimage candidate

$$\mathsf{pt}' := M_1^{-1}(a \oplus \gamma_1).$$

Equations (26)–(27), together with the recorded permutation evaluations, imply

$$\mathsf{Pylon}(\mathrm{iv}_{\mathrm{owf}}, \mathsf{pt}') = \mathsf{ct}.$$

Conversely, if $\mathcal{A}^+$ succeeds, its final evaluation queries expose a consistent pair. Writing CP for the event that such a pair occurs, we obtain

$$\mathsf{Adv}^{\mathrm{pre,IP}}_{\mathrm{Pylon}}(\mathcal{A}) = \mathsf{Adv}^{\mathrm{pre,IP}}_{\mathrm{Pylon}}(\mathcal{A}^+) \le \Pr[\mathsf{CP}].$$

Unique completing partner. For a fixed record $(a, y) \in \mathcal{T}_1$, define $p := M_1^{-1}(a \oplus \gamma_1)$. Its only possible completing second-permutation record is

$$c = M_2(y \oplus p) \oplus \mathrm{iv}_{\mathrm{owf}}, \qquad z = M_3^{-1}(\mathsf{ct} \oplus c \oplus p \oplus \gamma_2).$$

Conversely, a fixed record $(c, z) \in \mathcal{T}_2$ determines

$$\begin{cases} p = \mathsf{ct} \oplus M_3(z) \oplus c \oplus \gamma_2, \\ a = M_1(p) \oplus \gamma_1, \\ y = M_2^{-1}(c \oplus \mathrm{iv}_{\mathrm{owf}}) \oplus p. \end{cases}$$

Thus each record has at most one completing record on the other side. This is a deterministic property of the consistency equations. It does not, by itself, establish a probability bound for adaptive queries.

First-occurrence decomposition. For $i = 1, \ldots, Q$, let F_i be the event that no consistent pair exists before query i and that at least one consistent pair exists immediately after query i. If the adversary stops before query i, this event is false.

The events F_i are pairwise disjoint, and $\Pr[\mathsf{CP}] = \sum_{i=1}^{Q} \Pr[\mathsf{F}_i]$. This decomposition identifies the query that first completes a consistent pair, but does not yet bound its probability.

Challenge-path dependencies. Let (a^*, y^*) and (c^*, z^*) be the two permutation records used internally to generate the challenge. Let $\mathsf{HitPath}$ denote the event that a forward or inverse query of $\mathcal{A}^+$ exposes either of these records.

A valid decomposition is

$$\Pr[\mathsf{CP}] \leq \Pr[\mathsf{HitPath}] + \Pr[\mathsf{CP} \wedge \neg\mathsf{HitPath}].$$

Bounding these terms requires analysing the conditional distributions given the public value ct and the adaptive transcript. In particular, neither the uniformity of a hidden challenge-path point nor independence of fresh responses after conditioning on $\neg\mathsf{HitPath}$ follows merely from the initial independence of S_1 and S_2.

Remark 1 (Scope of the generic analysis). The consistent-pair characterization and first-occurrence decomposition above do not establish the previously proposed bound

$$\mathsf{Adv}^{\mathrm{pre,IP}}_{\mathrm{Pylon}}(\mathcal{A}) \leq \frac{3(q+2)}{2^n - q - 2}.$$

A proof of that bound would additionally have to control the challenge-path event and all four types of adaptive permutation queries under the correct conditional distributions. No such numerical bound is claimed as proved here. The signature reduction below therefore does not invoke this expression or infer a $\Theta(2^n)$ lower bound from it.

Remark 2 (Multi-user setting). Suppose u users share the same public permutations and other public parameters. Let Win_k denote successful inversion of target k in the experiment in which the adversary receives all u public targets. Then

$$\Pr\left[\bigcup_{k=1}^{u} \mathsf{Win}_k\right] \leq \sum_{k=1}^{u} \Pr[\mathsf{Win}_k].$$

Each probability on the right is evaluated in that same multi-target experiment. A single-target bound cannot be substituted without justifying the effect of the other targets, the shared permutations, and the key-generation distribution. Accordingly, no $O(uq/2^n)$ multi-user bound is deduced here.

6.6.3 EUF-CMA Security of GreatWall in the Random Oracle Model

We follow the FAEST/VOLEitH simulation-and-extraction methodology [9,8], together with the commitment and grinding design used in [6]. The relevant classical FAEST arguments analyse signing simulation and witness extraction through a sequence of games.

For GreatWall, these arguments must be instantiated for the actual commitment construction, constraint system, hash inputs, and rejection conditions. The theorem below is conditional on the explicitly stated simulation and extraction properties. It does not assert that these properties or their numerical losses follow automatically from the use of VOLEitH.

Key distribution and preimage experiment. Let $\mathcal{D}_{\mathrm{KG}}$ be the public-key distribution generated by the actual KeyGen algorithm, including its rejection conditions. Define

$$\mathsf{Adv}^{\mathrm{pre,KG}}_{\mathsf{Pylon}}(\mathcal{B}) := \Pr\big[\mathsf{Pylon}(\mathrm{iv}_{\mathrm{owf}}, pt') = ct\big],$$
$$(pk, sk) \leftarrow \mathsf{KeyGen}(1^\lambda), \qquad pk = (\mathrm{iv}_{\mathrm{owf}}, ct), \qquad pt' \leftarrow \mathcal{B}(pk).$$

The challenger does not give sk to $\mathcal{B}$.

The experiment uses the same public parameters and oracle interfaces as the signature experiment. If the SHAKE calls used in key generation are idealized as random-oracle calls, the oracle is sampled before key generation and retained consistently throughout the experiment. The reduction does not resample the public dot-product vectors or other key-generation oracle values.

This experiment concerns concrete Pylon under the KeyGen distribution. It is distinct from the unconditioned ideal-permutation experiment in Game 1.

Witness relation. The public statement is $pk = (\mathrm{iv}_{\mathrm{owf}}, ct)$, and the extended witness is $w = (pt, x_1)$. Define

$$s_1 = M_1(pt) \oplus \gamma_1, \qquad y_1 = x_1,$$
$$s_2 = M_2(x_1 \oplus pt) \oplus \mathrm{iv}_{\mathrm{owf}}, \quad y_2 = M_3^{-1}(ct \oplus s_2 \oplus \gamma_2 \oplus pt).$$

Let $\alpha_0^{\mathrm{dot}}, \alpha_1^{\mathrm{dot}}$ be the public n-bit vectors derived from $\mathrm{iv}_{\mathrm{owf}}$ as in key generation. The intended relation of the multiplication and dot-product constraints is

$$R_{\mathsf{GW}} := \big\{(pk, (pt, x_1)) : s_j y_j = y_j^{2^{e_j}}, \\ \langle \alpha_{j-1}^{\mathrm{dot}}, y_j \rangle = 1, \quad j = 1, 2\big\}.$$

The dot-product equations imply $y_j \neq 0$. Hence $s_j = y_j^{2^{e_j}-1}$. For the specified exponents, $\gcd(2^{e_j}-1, 2^n-1) = 1$, so the inverse Mersenne permutation gives $y_j = S_j(s_j)$. Substituting into the output equation yields $\mathsf{Pylon}(\mathrm{iv}_{\mathrm{owf}}, pt) = ct$. Thus a valid witness for R_{GW} supplies a valid Pylon preimage. The correctness of the specified prover and verifier for this relation must be checked separately.

Query accounting. Let q_S and q_H bound the adversary's signing and random-oracle queries, respectively. Let $\mathbf{Q}$ record the query counts for the distinct oracle domains in the complete reduction, including simulation, extraction, final verification, and grinding.

Write $\varepsilon_{\mathrm{sim}} = \varepsilon_{\mathrm{sim}}(q_S, \mathbf{Q}; \mathrm{par}), \varepsilon_{\mathrm{ks}} = \varepsilon_{\mathrm{ks}}(q_S, \mathbf{Q}; \mathrm{par})$, where par contains the complete protocol parameters, including $\tau, (N_i)_i, T_{\mathrm{open}}, w_{\mathrm{grind}}$ and the consistency-hash parameters.

Definition 2 (Required instantiation properties in the classical ROM). *The reduction requires the following properties of the complete GreatWall instantiation.*

1. ***Adaptive signing simulation.*** *There exists a PPT simulator* Sim *that, using pk alone and the current oracle interface, answers an adaptive sequence of signing queries. Replacing the real signing oracle by this simulator changes the adversary's winning probability by at most* $\varepsilon_{\mathrm{sim}}$.
   This guarantee concerns the complete simulated experiment, including consistent oracle programming, signature randomness, the counter, grinding, BAVC opening rejection, and any primitive-replacement hybrids required by the proof. Programming conflicts are checked before the corresponding oracle values are changed.
2. ***Straight-line extraction in the simulated experiment.*** *There exists a PPT straight-line extractor* Ext *integrated with the preceding oracle simulation, without changing the view supplied to the adversary, such that*

$$\Pr\big[\mathsf{Win}_{\mathrm{sim}} \ \wedge \ \big(w' = \perp \ \vee \ (pk, w') \notin R_{\mathsf{GW}}\big)\big] \leq \varepsilon_{\mathrm{ks}}.$$

   Here $\mathsf{Win}_{\mathrm{sim}}$ *means that the adversary outputs a valid signature on a message not submitted to the simulated signing oracle, and* w' *is the extractor's output.*
   This is an unconditional joint-event bound in the same simulated experiment. It is not a bound on extraction failure conditioned on an arbitrary accepting proof.
   The analysis must account for commitment extraction, VOLE consistency, constraint aggregation, and the hash-chain bad events involving programmed signing transcripts.

These properties are instantiation requirements. They are not asserted to follow from ordinary HVZK and stand-alone knowledge soundness without further analysis.

Hash-chain analysis required by the framework. For the signing algorithm as specified, $\mu = H_1(ct\|M; 2\lambda)$, and the Fiat–Shamir input strings are

$$\begin{aligned} X_1 &= \mu\|com\|c_1\|\cdots\|c_{\tau-1}\|\mathrm{iv}_{\mathrm{vole}}, \\ X_2 &= chall_1\|\widetilde{u}\|h_V\|d, \\ X_3 &= chall_2\|\widetilde{a}\|\widetilde{b}\|ctr. \end{aligned}$$

The FAEST-style bad-event analysis must show how a fresh message leads to an extractable forgery despite the programmed signing transcripts. This requires accounting for message-hash collisions, link collisions, guesses, and reused programmed inputs throughout the challenge chain.

It is insufficient to require the final challenge output to differ from all challenge outputs appearing in signatures. Moreover, the final signing challenge is selected subject to the grinding condition and a successful BAVC opening; it is not uniform over all λ-bit strings.

EUF-CMA experiment. The experiment $\mathsf{Exp}^{\mathsf{EUF\text{-}CMA}}_{\mathsf{GreatWall}}(\mathcal{A})$ samples

$$(pk^*, sk^*) \leftarrow \mathsf{KeyGen}(1^\lambda), \qquad pk^* = (\mathrm{iv}^*_{\mathrm{owf}}, ct^*).$$

It gives $\mathcal{A}$ the public key, access to the random oracles, and access to a signing oracle under sk^*. Let $\mathcal{Q}$ be the set of signing-query messages.

The adversary wins if it outputs (M^*, σ^*) such that $M^* \notin \mathcal{Q}, \mathsf{Verify}(M^*, pk^*, \sigma^*) = 1$. Then define it as

$$\mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathsf{GreatWall}}(\mathcal{A}) := \Pr[\mathcal{A} \text{ wins}].$$

Theorem 1 (Conditional ROM reduction to Pylon preimage resistance). *Suppose that the simulation and extraction properties in Definition 2 hold for the specified GreatWall algorithms and resource budgets. Then, for every PPT adversary $\mathcal{A}$, there exists a PPT preimage adversary $\mathcal{B}$ such that*

$$\mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathsf{GreatWall}}(\mathcal{A}) \le \mathsf{Adv}^{\mathrm{pre,KG}}_{\mathsf{Pylon}}(\mathcal{B}) + \varepsilon_{\mathrm{sim}} + \varepsilon_{\mathrm{ks}}.$$

The reduction runs $\mathcal{A}$ once and incurs the costs of the simulator, oracle handling, integrated extraction, verification, and relation checking.

Proof. We retain the original game-based simulation-and-extraction structure.

Game 2 (Real signing)**.** *This is the real EUF-CMA experiment. The key pair is generated by the actual* KeyGen *algorithm, and the signing oracle uses the secret key. Thus*

$$\Pr[\mathcal{A} \textit{ wins in Game 2}] = \mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathsf{GreatWall}}(\mathcal{A}).$$

Game 3 (Simulated signing)**.** *Replace the signing oracle by the simulator of Definition 2(1). The simulator uses pk^* and the current oracle state, but not sk^*. By the adaptive signing-simulation guarantee,*

$$\left|\Pr[\mathcal{A} \textit{ wins in Game 3}] - \Pr[\mathcal{A} \textit{ wins in Game 2}]\right| \le \varepsilon_{\mathrm{sim}}.$$

Game 4 (Extraction in the simulated experiment)**.** *Run Game 3 with the integrated extractor from Definition 2(2). The adversary's view is unchanged. The game returns success only if the adversary produces a fresh accepting forgery and the extractor outputs w' satisfying*

$$(pk^*, w') \in R_{\mathsf{GW}}.$$

Therefore,

$$0 \le \Pr[\mathcal{A} \textit{ wins in Game 3}] - \Pr[\textit{Game 4 succeeds}] \le \varepsilon_{\mathrm{ks}}.$$

There is no additional abort based solely on equality between final challenge outputs.

Reduction. Given a preimage challenge $pk^* = (\mathrm{iv}^*_{\mathrm{owf}}, ct^*)$ from the KeyGen-based experiment, $\mathcal{B}$ runs the simulated experiment and its integrated extractor. If Game 4 succeeds with $w' = (pt', x'_1)$, it returns pt'. Otherwise it returns $\perp$. The reduction interface is given in Algorithm 20.

Correctness of the reduction. The preimage challenge and the signature experiment use the same joint distribution of the public key and oracle environment. The simulated signing and extraction procedures are the ones required by Definition 2.

Whenever Game 4 succeeds, the extracted witness satisfies R_{GW}. The relation calculation above then gives $\mathsf{Pylon}(\mathrm{iv}^*_{\mathrm{owf}}, pt') = ct^*$. Hence $\Pr[\text{Game 4 succeeds}] \le \mathsf{Adv}^{\mathrm{pre,KG}}_{\mathsf{Pylon}}(\mathcal{B})$.

Combining. The game transitions imply

$$\begin{aligned}
\mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathsf{GreatWall}}(\mathcal{A}) &= \Pr[\mathcal{A} \text{ wins in Game 2}] \\
&\le \Pr[\mathcal{A} \text{ wins in Game 3}] + \varepsilon_{\mathrm{sim}} \\
&\le \Pr[\text{Game 4 succeeds}] + \varepsilon_{\mathrm{ks}} + \varepsilon_{\mathrm{sim}} \\
&\le \mathsf{Adv}^{\mathrm{pre,KG}}_{\mathsf{Pylon}}(\mathcal{B}) + \varepsilon_{\mathrm{ks}} + \varepsilon_{\mathrm{sim}}.
\end{aligned}$$

Running time. If T_{Sim} bounds the cost of one simulated signing query, the reduction's running time can be accounted for as

$$T(\mathcal{B}) \le T(\mathcal{A}) + q_S T_{\mathrm{Sim}} + T_{\mathrm{Ext}}(\mathbf{Q}) + T_{\mathrm{RO}}(\mathbf{Q}) + T_{\mathrm{Verify}} + T_{R_{\mathsf{GW}}}.$$

Here T_{Ext} includes the complete integrated extraction work. Straight-line execution does not by itself establish a linear bound in the number of oracle queries.

The PPT statement assumes polynomial-time simulation and extraction as required by Definition 2. If a procedure is only expected polynomial time because of rejection sampling, either an expected-time reduction must be stated or truncation and its additional failure probability must be included. □

Remark 3 (Concrete parameters and remaining obligations). Theorem 1 is a conditional reduction. It becomes a concrete security statement only after establishing the simulation and extraction properties for the exact GreatWall algorithms and giving their resource-dependent losses.

In particular, the field and exponent parameters in Table 2 alone do not imply $\varepsilon_{\mathrm{ks}} \leq 2^{-\lambda}$. The complete VOLEitH parameters, primitive-security terms, oracle budgets, grinding behaviour, and opening restrictions must be included.

Preimage resistance of concrete Pylon for the actual KeyGen distribution remains an assumption. Neither the ideal-permutation analysis nor the evaluation of known structural attacks is substituted for that assumption.

Algorithm 20: Conditional preimage reduction $\mathcal{B}(pk^*)$ in the ROM

Input: Preimage challenge $pk^* = (\mathrm{iv}^*_{\mathrm{owf}}, ct^*)$ from the KeyGen-based experiment, with the corresponding public oracle environment.
Output: pt' satisfying $\mathsf{Pylon}(\mathrm{iv}^*_{\mathrm{owf}}, pt') = ct^*$, or $\perp$.

$\mathcal{Q} \leftarrow \emptyset$;
Initialize the oracle simulation and integrated extractor consistently with the challenge experiment;
Index oracle records by (H, x), including the oracle domain;
Give pk^* to $\mathcal{A}$;
while $\mathcal{A}$ *has not output its forgery* **do**
  Process the next action of $\mathcal{A}$;
  if *the action is an oracle query* (H, x) **then**
    Answer through the consistent oracle interface;
    Update the records and run the prescribed extractor bookkeeping;
  else if *the action is a signing query* M **then**
    Run $\mathsf{Sim}(pk^*, M)$ using the current oracle and programming interfaces;
    Check compatibility before each programming operation;
    Handle simulation failures as prescribed by Definition 2(1);
    Obtain the complete simulated signature σ;
    $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{M\}$;
    Send σ to $\mathcal{A}$;
Receive (M^*, σ^*);
if $M^* \in \mathcal{Q}$ ***or*** $\mathsf{Verify}(M^*, pk^*, \sigma^*) \neq 1$ **then**
  return $\perp$;
Finalize the integrated extraction procedure to obtain w';
if $w' = \perp$ ***or*** $(pk^*, w') \notin R_{\mathsf{GW}}$ **then**
  return $\perp$;
Parse $w' = (pt', x'_1)$;
return pt';

6.6.4 Post-Quantum Security Framework and Scope

GreatWall is intended to resist quantum adversaries making classical signing queries and quantum queries to the random oracles. The ROM reduction in Theorem 1 does not establish this claim.

We retain the existing FAEST/VOLEitH methodology: analyse forgery extraction and adaptive signing simulation in the quantum-accessible oracle model, with compatible oracle interfaces and explicitly accounted losses.

Status of the cited framework. In the FAEST v1.0 specification [9], Section 10.1.3 gives a QROM signing-simulation result in Theorem 3. Its key-only security result, Theorem 5, relies on Conjecture 1 concerning quantum extraction from the VOLE commitment.

Accordingly, that reference does not by itself establish the required quantum extraction property for GreatWall's exact one-tree instantiation. The commitment construction, VOLE consistency check, constraint system, and challenge derivation must be shown to satisfy the hypotheses of the selected existing result.

The general Fiat–Shamir analyses of [17,29], the adaptive-reprogramming analysis of [22], and the compressed-oracle technique of [41] provide relevant tools. They are not interchangeable statements of online extractability for the complete GreatWall protocol.

Quantum signing simulation. Reprogramming must be analysed at the actual oracle input strings used by the signing procedure. The adaptive-reprogramming results of [22] depend on the query budget and the distributions of the reprogrammed points relative to the adversary's prior quantum interaction.

For GreatWall, the analysis must include the signing counter, grinding, opening rejection, and the full challenge chain. The classical expression $\frac{q_S q_H + q_S^2}{2^\lambda}$ is therefore not asserted as a QROM simulation bound. Nor is the classical union-bound analysis of individual oracle outputs reused without a quantum argument.

Quantum extraction and composition. A compressed-oracle representation does not alone supply an extractor, an extraction-error formula, or a particular running-time bound.

A quantum extraction analysis must apply to the complete GreatWall protocol and remain valid in the same experiment as the simulated signing oracle. Its error must account for accepting fresh forgeries that do not yield a valid witness, together with any disturbance or other losses introduced by the extraction procedure. These properties require an explicit instantiation of the chosen existing framework.

Quantum preimage assumption. The underlying assumption is quantum preimage resistance of concrete Pylon for the public-key distribution induced by the actual KeyGen algorithm, including its rejection conditions and associated oracle environment.

The attack estimates in Sect. 6.4 inform this assumption. They do not establish that every quantum algorithm attacking concrete Pylon has success probability $O(q_O^2/2^n)$. Grover search supplies a generic attack benchmark; it is not, by itself, a lower-bound proof for this specified algebraic construction.

Scope of the post-quantum claim. A complete QROM security theorem for GreatWall requires a compatible quantum signing-simulation and forgery-extraction analysis for the exact algorithms, with explicit assumptions, query accounting, and quantitative losses.

This section records the existing proof methodology and the remaining instantiation requirements. It does not claim the previously stated concrete QROM advantage or running-time bound, and it does not assert that GreatWall automatically inherits those bounds from another VOLEitH signature scheme.

7 Implementation and Performance

7.1 The Benchmarking Environments

The benchmarks were conducted on the same computing environment of Ubuntu 22.04.5 LTS. The platform is equipped with an Intel Core i9-14900HX @2.42 GHz and 15GB of RAM. All implementations were compiled with GCC 11.4.0, optimized implementations compiled with the AVX2 instructions. Our implementation is mainly referring to faest-one-tree repository[1]

7.2 Performance of GreatWall

As noted in the previous sections, the serial iterative structure of Pylon allows it to achieve its full potential when operating over large finite fields. In addition, the structure scales naturally to larger fields and no changes are required except for adjusting the parameters. The performance of Optimized GreatWall is summarized in Table 10 and the Reference version is in Table 11. The results show that the proposed construction achieves a noticeable reduction in signature size under larger-field instantiations, while maintaining practical signing and verification performance.

References

1. Bardet, M., Faugere, J.C., Salvy, B.: On the complexity of gröbner basis computation of semi-regular overdetermined algebraic equations. In: Proceedings of the International Conference on Polynomial System Solving. pp. 71–74 (2004)
2. Bardet, M., Faugere, J.C., Salvy, B., Yang, B.Y.: Asymptotic behaviour of the degree of regularity of semi-regular polynomial systems. In: Proc. of MEGA. vol. 5, pp. 2–2 (2005)

[1] https://github.com/faest-sign/faest-one-tree

Scheme	$(\tau, w, T_{\text{open}})$	pk (B)	KeyGen (ms)	Sign (ms)	Verify (ms)	Size (B)
GreatWall-128S	(11, 7, 100)	36	0.008	4.002	6.550	2758
GreatWall-128F	(16, 8, 108)	36	0.009	0.493	0.942	3396
GreatWall-160S	(14, 0, 160)	42	0.013	9.335	15.541	4976
GreatWall-160F	(20, 0, 160)	42	0.013	1.350	2.094	5720
GreatWall-192S	(16, 8, 183)	50	0.015	10.763	18.359	6804
GreatWall-192F	(24, 8, 184)	50	0.014	1.337	2.112	8012
GreatWall-256S	(22, 6, 246)	66	0.018	12.589	22.082	12236
GreatWall-256F	(32, 7, 248)	66	0.019	1.879	3.062	14260
GreatWall-512S	(44, 0, 512)	132	0.056	369.849	507.635	50012
GreatWall-512F	(64, 0, 512)	132	0.035	43.860	59.156	57812

Table 10: Performance of Optimized GreatWall for different parameter sets. The suffixes S and F denote slow and fast variants, respectively: S is optimized for shorter signatures, while F is optimized for faster signing and verification.

Scheme	$(\tau, w, T_{\text{open}})$	pk (B)	KeyGen (ms)	Sign (ms)	Verify (ms)	Size (B)
GreatWall-128S	(11, 7, 100)	36	0.971	130.018	109.783	2758
GreatWall-128F	(16, 8, 108)	36	1.069	27.708	21.036	3396
GreatWall-160S	(14, 0, 160)	42	1.874	293.844	272.686	4976
GreatWall-160F	(20, 0, 160)	42	1.791	68.857	50.803	5720
GreatWall-192S	(16, 8, 183)	50	2.459	389.516	361.676	6804
GreatWall-192F	(24, 8, 184)	50	2.295	88.702	61.545	8012
GreatWall-256S	(22, 6, 246)	66	4.810	730.283	655.604	12236
GreatWall-256F	(32, 7, 248)	66	4.401	208.326	138.344	14260
GreatWall-512S	(44, 0, 512)	132	17.945	5216.079	4439.361	50012
GreatWall-512F	(64, 0, 512)	132	14.643	1999.747	1220.867	57812

Table 11: Performance of the Reference implementation for different parameter sets of GreatWall .

3. Bariant, A., Boeuf, A., Briaud, P., Hostettler, M., Øygarden, M., Raddum, H.: Improved resultant attack against arithmetization-oriented primitives. In: Annual International Cryptology Conference. pp. 335–367. Springer (2025)
4. Bariant, A., Boeuf, A., Lemoine, A., Manterola Ayala, I., Øygarden, M., Perrin, L., Raddum, H.: The algebraic freelunch: Efficient gröbner basis attacks against arithmetization-oriented primitives. In: Annual International Cryptology Conference. pp. 139–173. Springer (2024)
5. Bariant, A., Bouvier, C., Leurent, G., Perrin, L.: Algebraic attacks against some arithmetization-oriented primitives. IACR Transactions on Symmetric Cryptology pp. 73–101 (2022)
6. Baum, C., Beullens, W., Mukherjee, S., Orsini, E., Ramacher, S., Rechberger, C., Roy, L., Scholl, P.: One tree to rule them all: optimizing ggm trees and owfs for post-quantum signatures. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 463–493. Springer (2024)
7. Baum, C., Braun, L., Munch-Hansen, A., Scholl, P.: MozZ2karella: Efficient vector-ole and zero-knowledge proofs over Z2k. IACR Cryptol. ePrint Arch. **2022**, 819 (2022)
8. Baum, C., Braun, L., de Saint Guilhem, C.D., Klooß, M., Orsini, E., Roy, L., Scholl, P.: Publicly verifiable zero-knowledge and post-quantum signatures from vole-in-the-head. In: Annual international cryptology conference. pp. 581–615. Springer (2023)
9. Baum, C., Braun, L., de Saint Guilhem, C.D., Klooß, M., Majenz, C., Mukherjee, S., Ramacher, S., Rechberger, C., Orsini, E., Roy, L., et al.: Faest: Algorithm specifications. Tech. rep., National Institute of Standards and Technology (2023)
10. Bernstein, D.J., Yang, B.Y.: Asymptotically faster quantum algorithms to solve multivariate quadratic equations. In: International Conference on Post-Quantum Cryptography. pp. 487–506. Springer (2018)
11. Bonnetain, X., Hosoyamada, A., Naya-Plasencia, M., Sasaki, Y., Schrottenloher, A.: Quantum attacks without superposition queries: the offline simon's algorithm. In: International conference on the theory and application of cryptology and information security. pp. 552–583. Springer (2019)
12. Bosma, W., Cannon, J., Playoust, C.: The magma algebra system i: The user language. Journal of Symbolic Computation **24**(3-4), 235–265 (1997)
13. Carlitz, L.: A note on the betti-mathieu group. Portugaliae mathematica **22**, 121–125 (1963)
14. Courtois, N.T., Pieprzyk, J.: Cryptanalysis of block ciphers with overdefined systems of equations. In: International conference on the theory and application of cryptology and information security. pp. 267–287. Springer (2002)
15. Cox, D., Little, J., O'shea, D., Sweedler, M.: Ideals, varieties, and algorithms. Springer (1997)
16. Dobraunig, C., Kales, D., Rechberger, C., Schofnegger, M., Zaverucha, G.: Shorter signatures based on tailor-made minimalist symmetric-key crypto. In: Proceedings of the 2022 acm sigsac conference on computer and communications security. pp. 843–857 (2022)
17. Don, J., Fehr, S., Majenz, C., Schaffner, C.: Security of the fiat-shamir transformation in the quantum random-oracle model. In: Advances in Cryptology - CRYPTO 2019, Part II. Lecture Notes in Computer Science, vol. 11693, pp. 356–383. Springer (2019). https://doi.org/10.1007/978-3-030-26951-7_13
18. Faugere, J.C., Horan, K., Kahrobaei, D., Kaplan, M., Kashefi, E., Perret, L.: Fast quantum algorithm for solving multivariate quadratic equations. arXiv preprint arXiv:1712.07211 (2017)

19. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Conference on the theory and application of cryptographic techniques. pp. 186–194. Springer (1986)
20. Genovese, G.: Improving the algorithms of berlekamp and niederreiter for factoring polynomials over finite fields. Journal of Symbolic Computation **42**(1-2), 159–177 (2007)
21. Goldreich, O., Goldwasser, S., Micali, S.: How to construct random functions. Journal of the ACM (JACM) **33**(4), 792–807 (1986)
22. Grilo, A.B., Hövelmanns, K., Hülsing, A., Majenz, C.: Tight adaptive reprogramming in the QROM. In: Advances in Cryptology - ASIACRYPT 2021, Part I. Lecture Notes in Computer Science, vol. 13090, pp. 637–667. Springer (2021). https://doi.org/10.1007/978-3-030-92062-3_22
23. Grover, L.K.: A fast quantum mechanical algorithm for database search. In: Proceedings of the twenty-eighth annual ACM symposium on Theory of computing. pp. 212–219 (1996)
24. Kim, S., Ha, J., Son, M., Lee, B.: Efficacy and mitigation of the cryptanalysis on aim. Cryptology ePrint Archive (2023)
25. Kim, S., Ha, J., Son, M., Lee, B., Moon, D., Lee, J., Lee, S., Kwon, J., Cho, J., Yoon, H., et al.: Aim: symmetric primitive for shorter signatures with stronger security. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. pp. 401–415 (2023)
26. Lee, J., Cho, A.J., Kim, S., Kwon, J., Lee, B., Lee, J., Lee, S., Moon, D., Son, M., Yoon, H., et al.: The aimer signature scheme
27. Lidl, R., Niederreiter, H.: Finite fields. No. 20, Cambridge university press (1997)
28. Liu, F., Mahzoun, M., Meier, W.: Modelling ciphers with overdefined systems of quadratic equations: Application to friday, vision, rain and biscuit. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 424–456. Springer (2024)
29. Liu, Q., Zhandry, M.: Revisiting post-quantum fiat-shamir. In: Advances in Cryptology - CRYPTO 2019, Part II. Lecture Notes in Computer Science, vol. 11693, pp. 326–355. Springer (2019). https://doi.org/10.1007/978-3-030-26951-7_12
30. NIST: Sha-3 standard: Permutation-based hash and extendable-output functions (2015)
31. Rompel, J.: One-way functions are necessary and sufficient for secure signatures. In: Proceedings of the twenty-second annual ACM symposium on Theory of computing. pp. 387–394 (1990)
32. Shor, P.W.: Algorithms for quantum computation: discrete logarithms and factoring. In: Proceedings 35th annual symposium on foundations of computer science. pp. 124–134. Ieee (1994)
33. Sun, Y., Chen, S., Liu, G., Wang, M., Niu, C.: High exponents may not suffice to patch aim (on attacks, weak parameters, and patches for aim2). Cryptology ePrint Archive (2025)
34. Vazirani, U.: On the power of quantum computation. Philosophical Transactions of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences **356**(1743), 1759–1768 (1998)
35. Von Zur Gathen, J., Gerhard, J.: Modern computer algebra. Cambridge university press (2003)
36. Weng, C., Yang, K., Katz, J., Wang, X.: Wolverine: Fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits. In: 2021 IEEE Symposium on Security and Privacy (SP). pp. 1074–1091. IEEE (2021)

37. Wu, B., Liu, Z.: Linearized polynomials over finite fields revisited. Finite Fields and Their Applications **22**, 79–100 (2013)
38. Yang, H.S., Zheng, Q.X., Yang, J.: Algebraic cryptanalysis of ao primitives based on polynomial decomposition: Applications to rain and full aim-i/iii/v. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 383–410. Springer (2025)
39. Yang, H.S., Zheng, Q.X., Yang, J., Liu, Q.f., Tang, D.: A new security evaluation method based on resultant for arithmetic-oriented algorithms. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 457–489. Springer (2024)
40. Yang, K., Sarkar, P., Weng, C., Wang, X.: Quicksilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In: Proceedings of the 2021 ACM SIGSAC conference on computer and communications security. pp. 2986–3001 (2021)
41. Zhandry, M.: How to record quantum queries, and applications to quantum indifferentiability. In: Advances in Cryptology - CRYPTO 2019, Part II. Lecture Notes in Computer Science, vol. 11693, pp. 239–268. Springer (2019). https://doi.org/10.1007/978-3-030-26951-7_9
42. Zhandry, M.: How to construct quantum random functions. Journal of the ACM (JACM) **68**(5), 1–43 (2021)
43. Zhang, K., Wang, Q., Yu, Y., Guo, C., Cui, H.: Algebraic attacks on round-reduced rain and full aim-iii. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 285–310. Springer (2023)